

EE 269: Foundation Models and Advances in AI

Lecture 11: RL and Agents III

Yinglun Zhu
University of California, Riverside



Agentic RL

Agentic Reinforcement Learning (Agentic RL) refers to a paradigm in which LLMs, rather than being treated as *static conditional generators* optimized for single-turn output alignment or benchmark performance, are conceptualized as *learnable policies* embedded within sequential decision-making loops, where RL endows them with autonomous agentic capabilities, such as planning, reasoning, tool use, memory maintenance, and self-reflection, enabling the emergence of long-horizon cognitive and interactive behaviors in *partially observable, dynamic environments*.

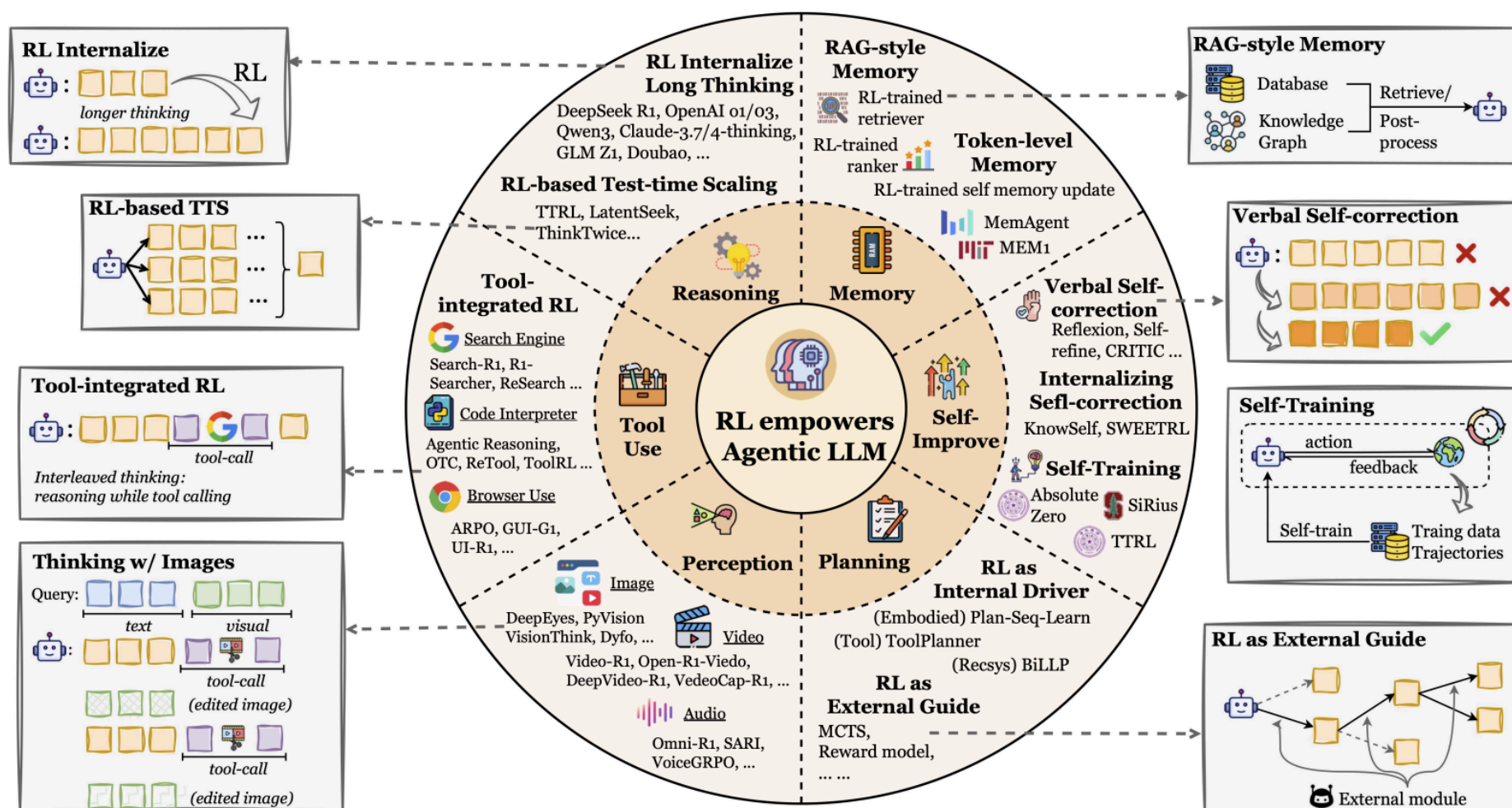


Figure 5: A conceptual overview of how RL empowers agentic LLMs across six core capabilities. The central panel summarizes the capability taxonomy, while the side panels illustrate representative RL mechanisms and interaction patterns. Listed methods are illustrative rather than exhaustive; see the main text for details.

Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning

Bowen Jin¹, Hansi Zeng², Zhenrui Yue¹, Jinsung Yoon³, Sercan Ö. Arık³, Dong Wang¹,
Hamed Zamani², Jiawei Han¹

¹ Department of Computer Science, University of Illinois at Urbana-Champaign

² Center for Intelligent Information Retrieval, University of Massachusetts Amherst

³ Google Cloud AI Research

{bowenj4, zhenrui3, dwang24, hanj}@illinois.edu, {hzeng, zamani}@cs.umass.edu
{jinsungyoon, soarik}@google.com

Abstract

Efficiently acquiring external knowledge and up-to-date information is essential for effective reasoning and text generation in large language models (LLMs). Prompting advanced LLMs with reasoning capabilities to use search engines during inference is often suboptimal, as the LLM might not fully possess the capability on how to interact optimally with the search engine. This paper introduces SEARCH-R1, an extension of reinforcement learning (RL) for reasoning frameworks where the LLM learns to autonomously generate (multiple) search queries during step-by-step reasoning with real-time retrieval. SEARCH-R1 optimizes LLM reasoning trajectories with multi-turn search interactions, leveraging retrieved token masking for stable RL training and a simple outcome-based reward function. Experiments on seven question-answering datasets show that SEARCH-R1 improves performance by 24% (Qwen2.5-7B) and 20% (Qwen2.5-3B) over various RAG baselines under the same setting. This paper further provides empirical insights into RL optimization methods, LLM choices, and response length dynamics in retrieval-augmented reasoning. The code and model checkpoints are available at <https://github.com/PeterGriffinJin/Search-R1>.

Search R1

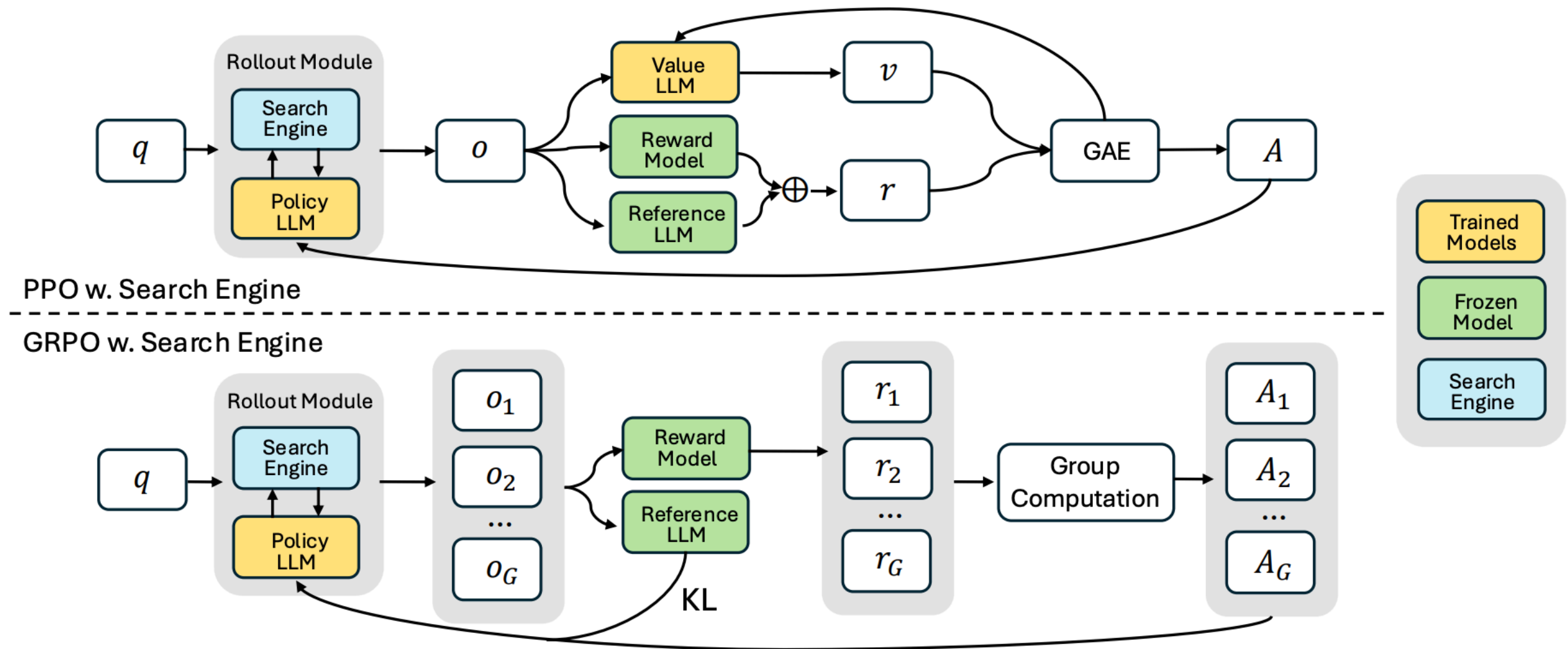


Figure 1: Demonstration of PPO and GRPO training with the search engine (SEARCH-R1). During the rollout, LLMs can conduct multi-turn interactions with the search engine.

Search R1

Answer the given question. You must conduct reasoning inside `<think>` and `</think>` first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<search>` query `</search>`, and it will return the top searched results between `<information>` and `</information>`. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>` without detailed illustrations. For example, `<answer>` xxx `</answer>`. Question: **question**.

Table 1: Template for SEARCH-R1. **question** will be replaced with the specific question during training and inference.

Search R1

3.4 Reward Modeling

The reward function serves as the primary training signal, guiding the optimization process in RL. To train SEARCH-R1, we adopt a rule-based reward system that consists solely of **final outcome rewards**, which assess the correctness of the model’s response. For instance, in factual reasoning tasks, correctness can be evaluated using rule-based criteria such as exact string matching:

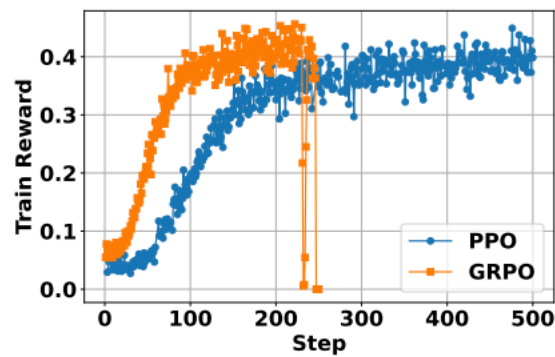
$$r_{\phi}(x, y) = \text{EM}(a_{\text{pred}}, a_{\text{gold}}), \quad (4)$$

Search R1

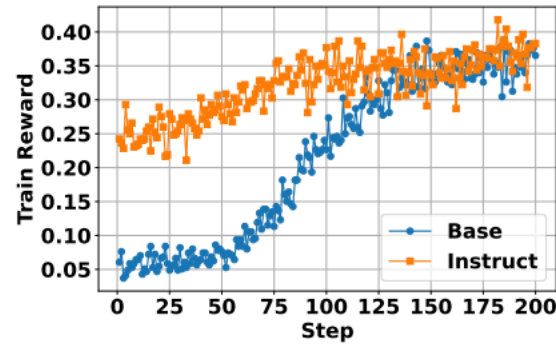
Table 2: Main results. The best performance is set in bold. [†]/^{*} represents in-domain/out-domain datasets.

Methods	General QA				Multi-Hop QA			
	NQ [†]	TriviaQA [*]	PopQA [*]	HotpotQA [†]	2wiki [*]	Musique [*]	Bamboogle [*]	Avg.
Qwen2.5-7b-Base/Instruct								
Direct Inference	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.181
CoT	0.048	0.185	0.054	0.092	0.111	0.022	0.232	0.106
IRCoT	0.224	0.478	0.301	0.133	0.149	0.072	0.224	0.239
Search-o1	0.151	0.443	0.131	0.187	0.176	0.058	0.296	0.206
RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.304
SFT	0.318	0.354	0.121	0.217	0.259	0.066	0.112	0.207
R1-base	0.297	0.539	0.202	0.242	0.273	0.083	0.296	0.276
R1-instruct	0.270	0.537	0.199	0.237	0.292	0.072	0.293	0.271
Rejection Sampling	0.360	0.592	0.380	0.331	0.296	0.123	0.355	0.348
Search-R1-base	0.480	0.638	0.457	0.433	0.382	0.196	0.432	0.431
Search-R1-instruct	0.393	0.610	0.397	0.370	0.414	0.146	0.368	0.385
Qwen2.5-3b-Base/Instruct								
Direct Inference	0.106	0.288	0.108	0.149	0.244	0.020	0.024	0.134
CoT	0.023	0.032	0.005	0.021	0.021	0.002	0.000	0.015
IRCoT	0.111	0.312	0.200	0.164	0.171	0.067	0.240	0.181
Search-o1	0.238	0.472	0.262	0.221	0.218	0.054	0.320	0.255
RAG	0.348	0.544	0.387	0.255	0.226	0.047	0.080	0.270
SFT	0.249	0.292	0.104	0.186	0.248	0.044	0.112	0.176
R1-base	0.226	0.455	0.173	0.201	0.268	0.055	0.224	0.229
R1-instruct	0.210	0.449	0.171	0.208	0.275	0.060	0.192	0.224
Rejection Sampling	0.294	0.488	0.332	0.240	0.233	0.059	0.210	0.265
Search-R1-base	0.406	0.587	0.435	0.284	0.273	0.049	0.088	0.303
Search-R1-instruct	0.341	0.545	0.378	0.324	0.319	0.103	0.264	0.325

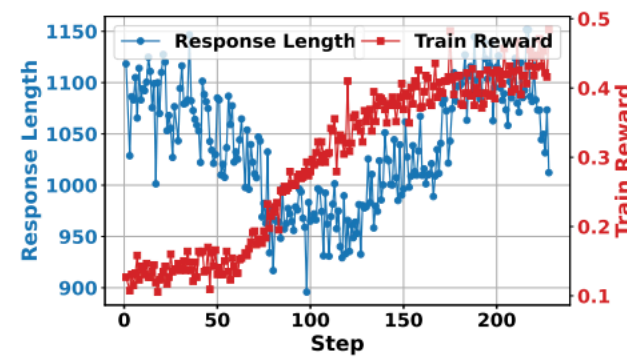
Search R1



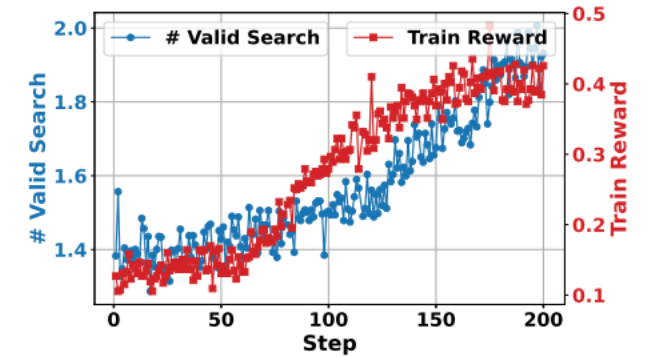
(a) PPO vs. GRPO



(b) Base vs. Instruct



(c) Response length



(d) # Valid search

Figure 2: (a) PPO vs. GRPO: GRPO generally converges faster but may exhibit instability after trained for a number of steps, whereas PPO provides more stable optimization but converges at a slower rate. (b) Base vs. Instruct LLM study: Instruction-tuned LLMs converge faster, but the final performance of both models remains highly similar. (c) Response length study: The response length exhibits a decrease-increase-stabilize trend throughout training, aligning with the overall performance trajectory of the LLM. (d) # Valid search study: As the training proceeds, the LLM learns to call search more.

Search R1

Table 4: The performance of SEARCH-R1 with and without retrieved token loss masking. The LLM trained with retrieved token loss masking achieves consistently better performance. (LLM: Qwen2.5-7b-base; RL: PPO)

Method	NQ	TriviaQA	PopQA	HotpotQA	2wiki	Musique	Bamboogle	Avg.
SEARCH-R1 w. mask	0.480	0.638	0.457	0.433	0.382	0.196	0.432	0.431
SEARCH-R1 w.o. mask	0.388	0.567	0.391	0.325	0.321	0.108	0.304	0.343

Loss Masking for Retrieved Tokens. In both PPO and GRPO, the token-level losses are computed over the entire rollout sequence. In SEARCH-R1, the rollout sequence consists of both LLM-generated tokens and retrieved tokens from external passages. While optimizing LLM-generated tokens enhances the model’s ability to interact with the search engine and perform reasoning, applying the same optimization to retrieved tokens can lead to unintended learning dynamics. To address this, we introduce loss masking for retrieved tokens, ensuring the policy gradient objective is computed only over LLM-generated tokens, excluding retrieved content from the optimization process. This approach stabilizes training while preserving the flexibility of search-augmented generation.

Training Configs

Parameter	DR.GRPO	DAPO	JustRL	Search-R1
Algorithm	DR.GRPO	DAPO	GRPO	GRPO
Model size	1.5B, 3B, 7B	32B	1.5B, 7B	3B, 7B, 14B
Temperature	1.0	1.0	1.0	1.0
Top-p (train / eval)	1.0 / —	1.0 / 0.7	1.0 / —	1.0 / —
Rollouts/prompt	8	16	8	5
Learning rate	1e-6	1e-6	1e-6	1e-6
LR warmup steps	0	10	100	28.5% of total
Weight decay	0.0	0.1	0.01	0.01
Gradient clip	1.0	1.0	1.0	1.0
KL coefficient	0 (disabled)	0 (disabled)	0 (disabled)	0.001
Entropy coefficient	0 (disabled)	0 (disabled)	0 (disabled)	0 (disabled)
$\epsilon_{\text{clip_low}}$	0.2	0.2	0.2	0.2
$\epsilon_{\text{clip_high}}$	0.2 (symmetric)	0.28	0.28	0.2 (symmetric)
Train prompt batch	128	512	256	512
Mini-batch size	128	32	64	256
Loss reduction	token-mean (unbiased)	token-mean	token-mean	token-mean

DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com

Abstract

We present a preview version of DeepSeek-V4 series, including two strong Mixture-of-Experts (MoE) language models — DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated) — both supporting a context length of one million tokens. DeepSeek-V4 series incorporate several key upgrades in architecture and optimization: (1) a hybrid attention architecture that combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) to improve long-context efficiency; (2) Manifold-Constrained Hyper-Connections (mHC) that enhance conventional residual connections; (3) and the Muon optimizer for faster convergence and greater training stability. We pre-train both models on more than 32T diverse and high-quality tokens, followed by a comprehensive post-training pipeline that unlocks and further enhances their capabilities. DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models, outperforming its predecessors in core tasks. Meanwhile, DeepSeek-V4 series are highly efficient in long-context scenarios. In the one-million-token context setting, DeepSeek-V4-Pro requires only 27% of single-token inference FLOPs and 10% of KV cache compared with DeepSeek-V3.2. This enables us to routinely support one-million-token contexts, thereby making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at <https://huggingface.co/collections/deepseek-ai/deepseek-v4>.

DeepSeek V4

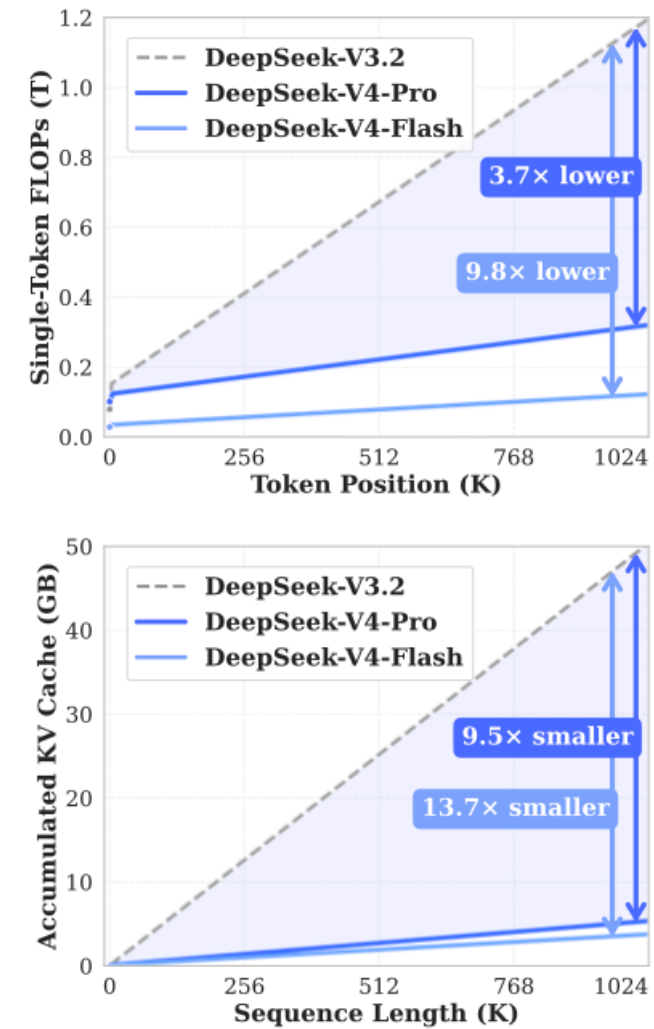
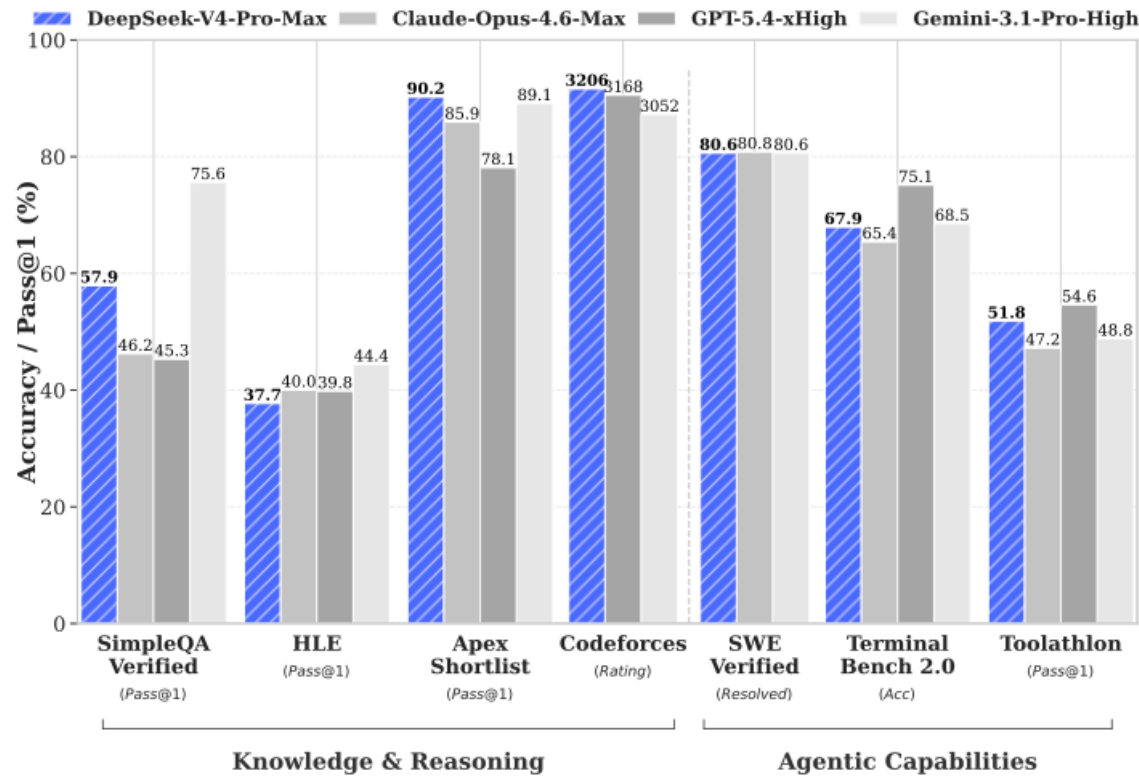


Figure 1 | **Left:** benchmark performance of DeepSeek-V4-Pro-Max and its counterparts. **Right:** inference FLOPs and KV cache size of DeepSeek-V4 series and DeepSeek-V3.2.

DeepSeek V4

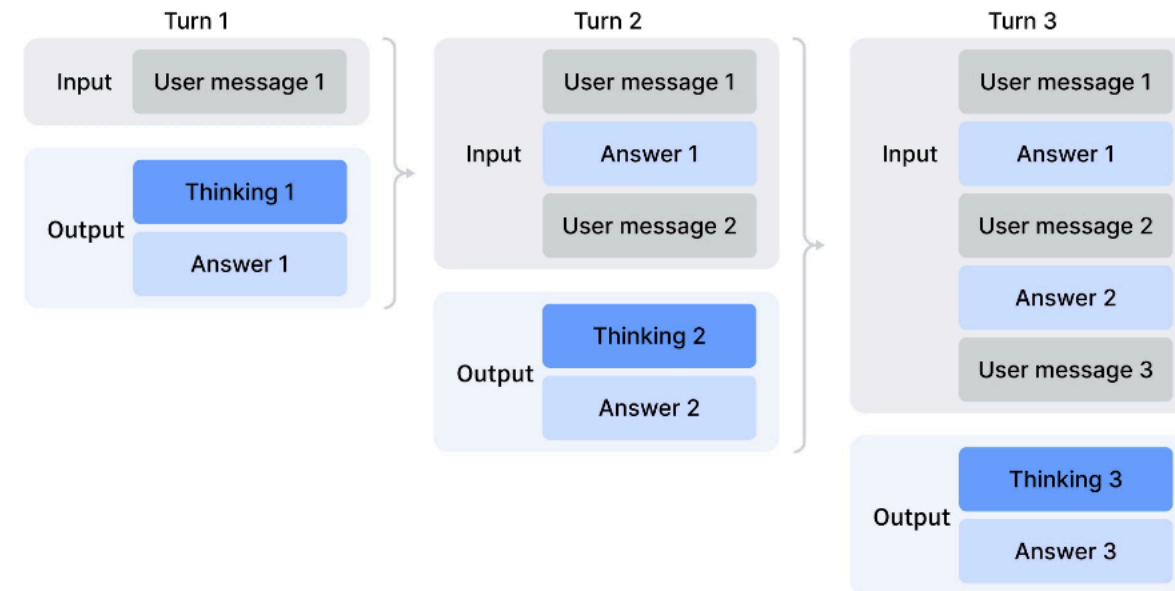
Post-training:

- Stage 1: SFT + RL for domain experts
 - Coding, Math, instruction-following, etc.
 - Generative reward model for hard-to-verify tasks
- Stage 2: On-policy distillation from experts
 - OPD to merge expert capabilities

DeepSeek V4



a) Thinking with tools



b) Thinking without tools

Figure 7 | Thinking management of DeepSeek-V4 series.

DeepSeek V4

$\{\pi_{E_1}, \pi_{E_2}, \dots, \pi_{E_N}\}$, the OPD objective function is defined as:

$$\mathcal{L}_{\text{OPD}}(\theta) = \sum_{i=1}^N w_i \cdot D_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i}). \quad (29)$$

In this formulation, w_i represents the assigned weight for each expert, typically determined by the relative importance of the expert. Computing the reverse KL loss $D_{\text{KL}}(\pi_{\theta} \parallel \pi_{E_i})$ requires

Gaussian Example

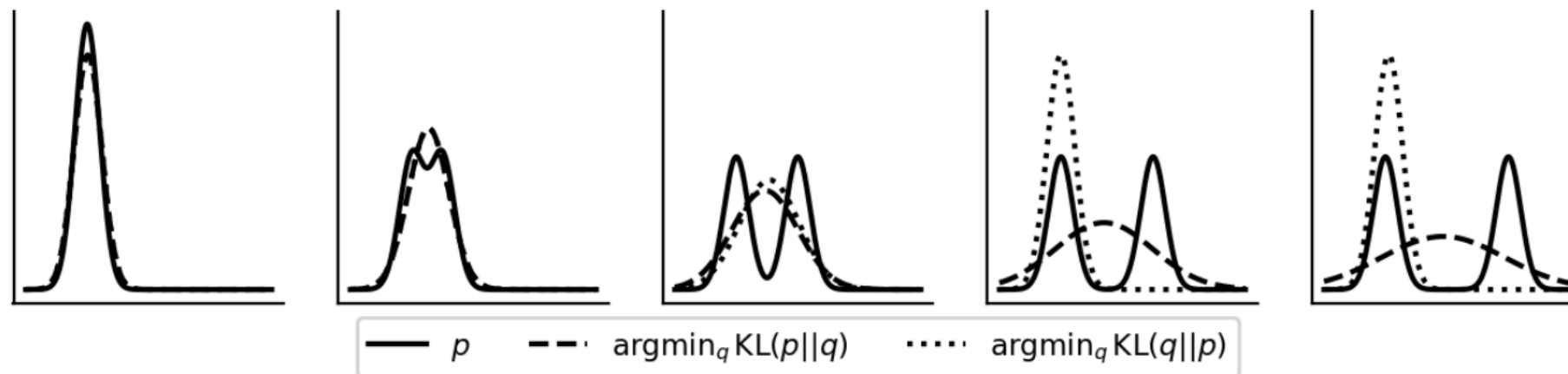
Let

$$p(x) = \sum_{k=1}^K \pi_k \text{Normal}(x; \mu_k, \sigma_k^2) \quad (1)$$

$$q_{\phi}(x) = \text{Normal}(x; \mu_q, \sigma_q^2), \quad (2)$$

where $K = 2$, $\pi_k = 0.5$, $\sigma_k^2 = 1$ ($k = 1, 2$), $\mu_1 = 0$, μ_2 is increasing from 0 to 10 and $\phi = (\mu_q, \sigma_q^2)$.

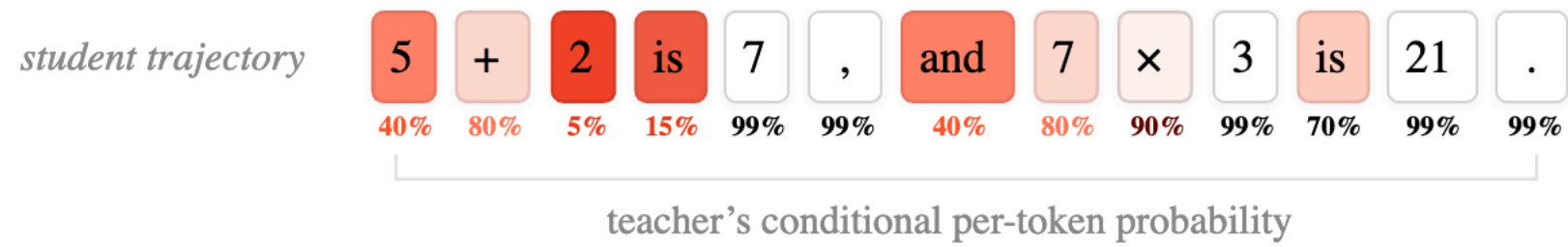
The behavior of minimizing the forward and reverse KL divergences with respect to q_{ϕ} is as follows:



[Python script for generating this figure.](#)

Intuitive explanations and speculations for the origins of the various terms:

On Policy Distillation



Method	Sampling	Reward signal
■ Supervised finetuning	off-policy	<u>dense</u>
■ Reinforcement learning	<u>on-policy.</u>	sparse
■ On-policy distillation	<u>on-policy.</u>	<u>dense</u>

On Policy Distillation

Choose the most realistic answer. Beth places four whole ice cubes in a frying pan at the start of the first minute, then five at the start of the second minute and some more at the start of the third minute, but none in the fourth minute. If the average number of ice cubes per minute placed in the pan while it was frying a crispy egg was five, how many whole ice cubes can be found in the pan at the end of the third minute?

A. 30 B. 0 C. 20 D. 10 E. 11 F. 5

(Start of assistant response truncated)

Step 3: Total ice cubes in the pan ****at the end of the third minute****

This means all ice cubes placed ****by the end of minute 3**** — that is, minute 1, 2, and 3.

So:

$4 \text{ (min 1)} + 5 \text{ (min 2)} + 11 \text{ (min 3)} = \textbf{**20**}$

Note: The ice cubes are not removed during the process — the pan is "frying a crispy egg", but we are only counting how many were added. It seems the ice cubes stay in the pan after being added.

Based on the context, we assume that ice cubes are not melted or removed during the frying process — otherwise we would have to account for melting, which is not mentioned.

Therefore, the total number of ice cubes ****in the pan at the end of the third minute**** is:

$4 + 5 + 11 = \textbf{**20**}$

Final Answer:

****C. 20****

****Correct answer: C. 20****<|im_end|>

Figure 6: Example trajectory graded by a teacher model. Tokens in darker red correspond to a higher reverse KL.

On Policy Distillation

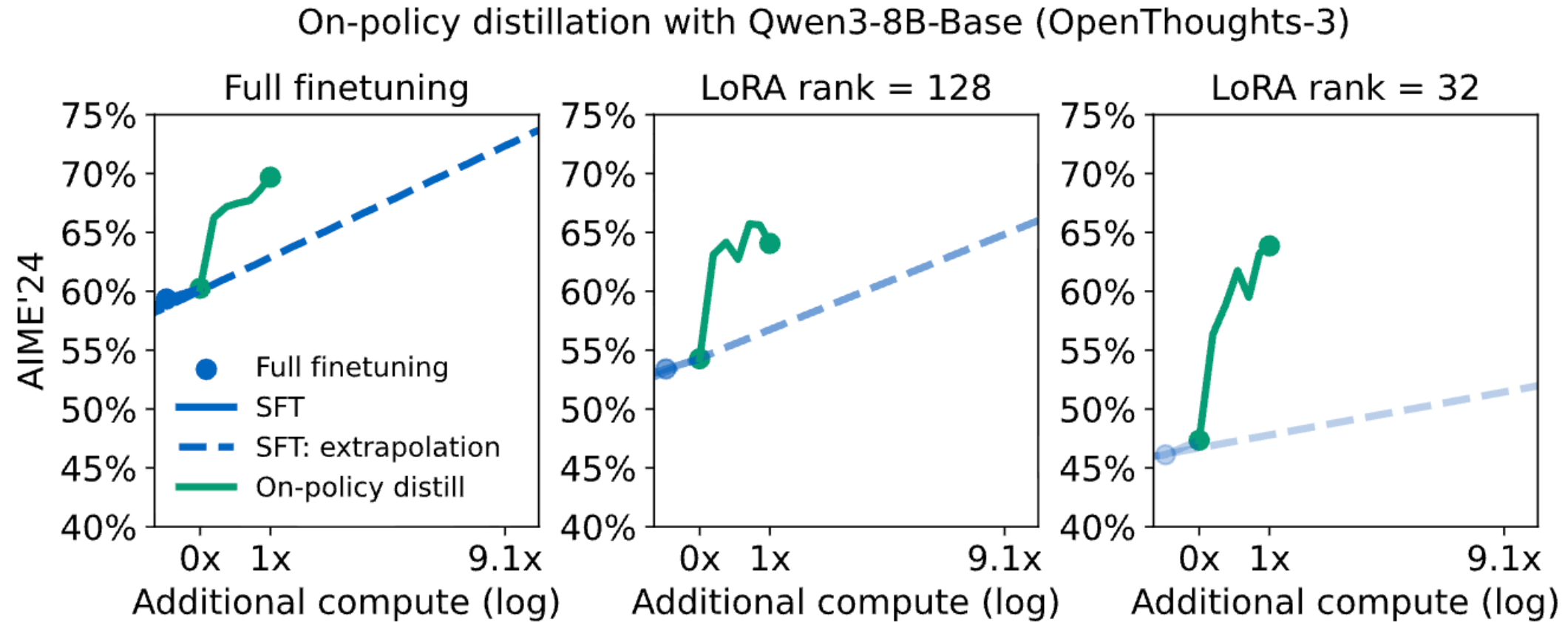


Figure 8: AIME'24 over the course of on-policy distillation. We measure additional compute in terms of training FLOPs (see below). On-policy distillation is significantly more compute-efficient than SFT, especially for LoRA models. At rank = 32, LoRA trails full finetuning by 13% after SFT, but only 6% after on-policy distillation.

On Policy Distillation

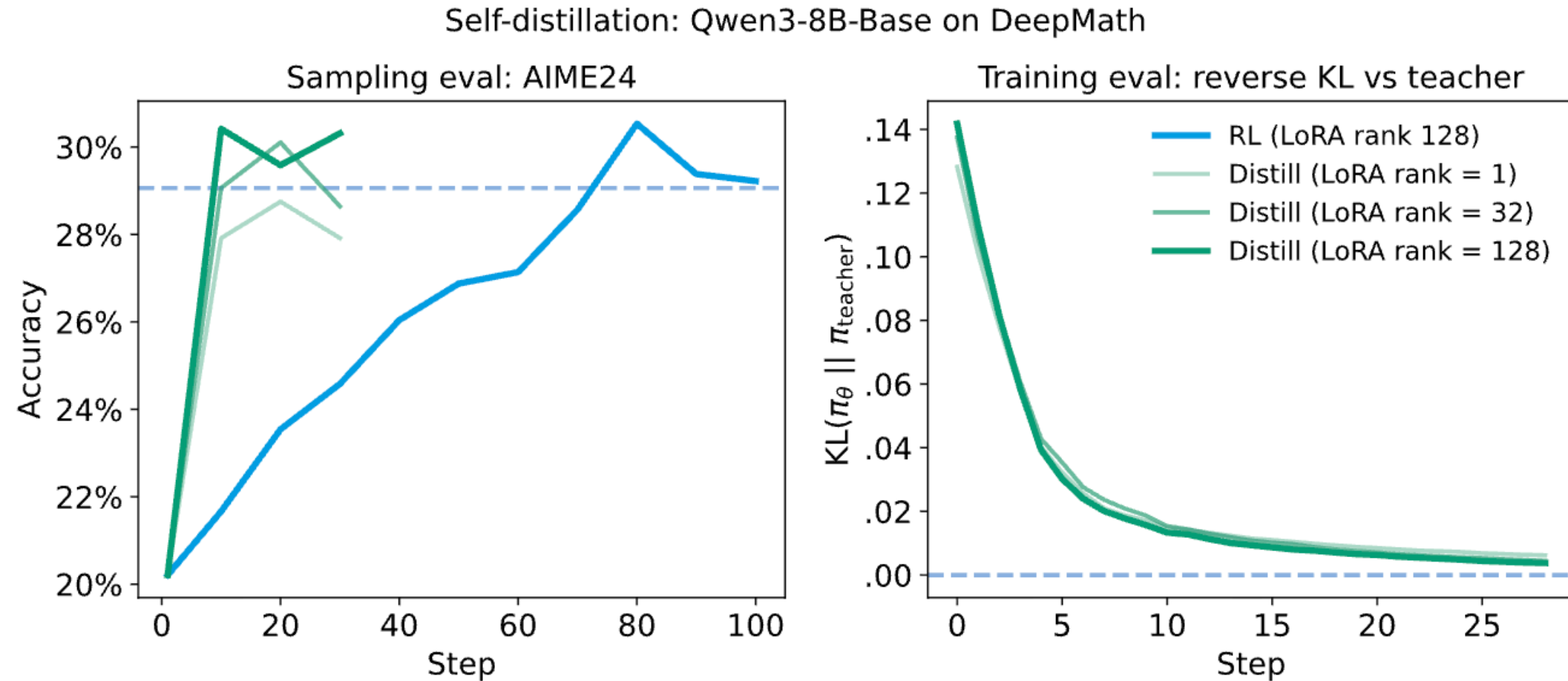


Figure 12: Starting from the same initialization, on-policy distillation can learn the RL-trained policy in approximately 7-10x fewer gradient steps, which corresponds to a compute efficiency of 50-100x.

KIMI K2: OPEN AGENTIC INTELLIGENCE

TECHNICAL REPORT OF KIMI K2

Kimi Team

ABSTRACT

We introduce Kimi K2, a Mixture-of-Experts (MoE) large language model with 32 billion activated parameters and 1 trillion total parameters. We propose the MuonClip optimizer, which improves upon Muon with a novel QK-clip technique to address training instability while enjoying the advanced token efficiency of Muon. Based on MuonClip, K2 was pre-trained on 15.5 trillion tokens with zero loss spike. During post-training, K2 undergoes a multi-stage post-training process, highlighted by a large-scale agentic data synthesis pipeline and a joint reinforcement learning (RL) stage, where the model improves its capabilities through interactions with real and synthetic environments.

Kimi K2 achieves state-of-the-art performance among open-source non-thinking models, with strengths in agentic capabilities. Notably, K2 obtains 66.1 on Tau2-Bench, 76.5 on ACEBench (En), 65.8 on SWE-Bench Verified, and 47.3 on SWE-Bench Multilingual — surpassing most open and closed-sourced baselines in non-thinking settings. It also exhibits strong capabilities in coding, mathematics, and reasoning tasks, with a score of 53.7 on LiveCodeBench v6, 49.5 on AIME 2025, 75.1 on GPQA-Diamond, and 27.1 on OJBench, all without extended thinking. These results position Kimi K2 as one of the most capable open-source large language models to date, particularly in software engineering and agentic tasks. We release our base and post-trained model checkpoints¹ to facilitate future research and applications of agentic intelligence.

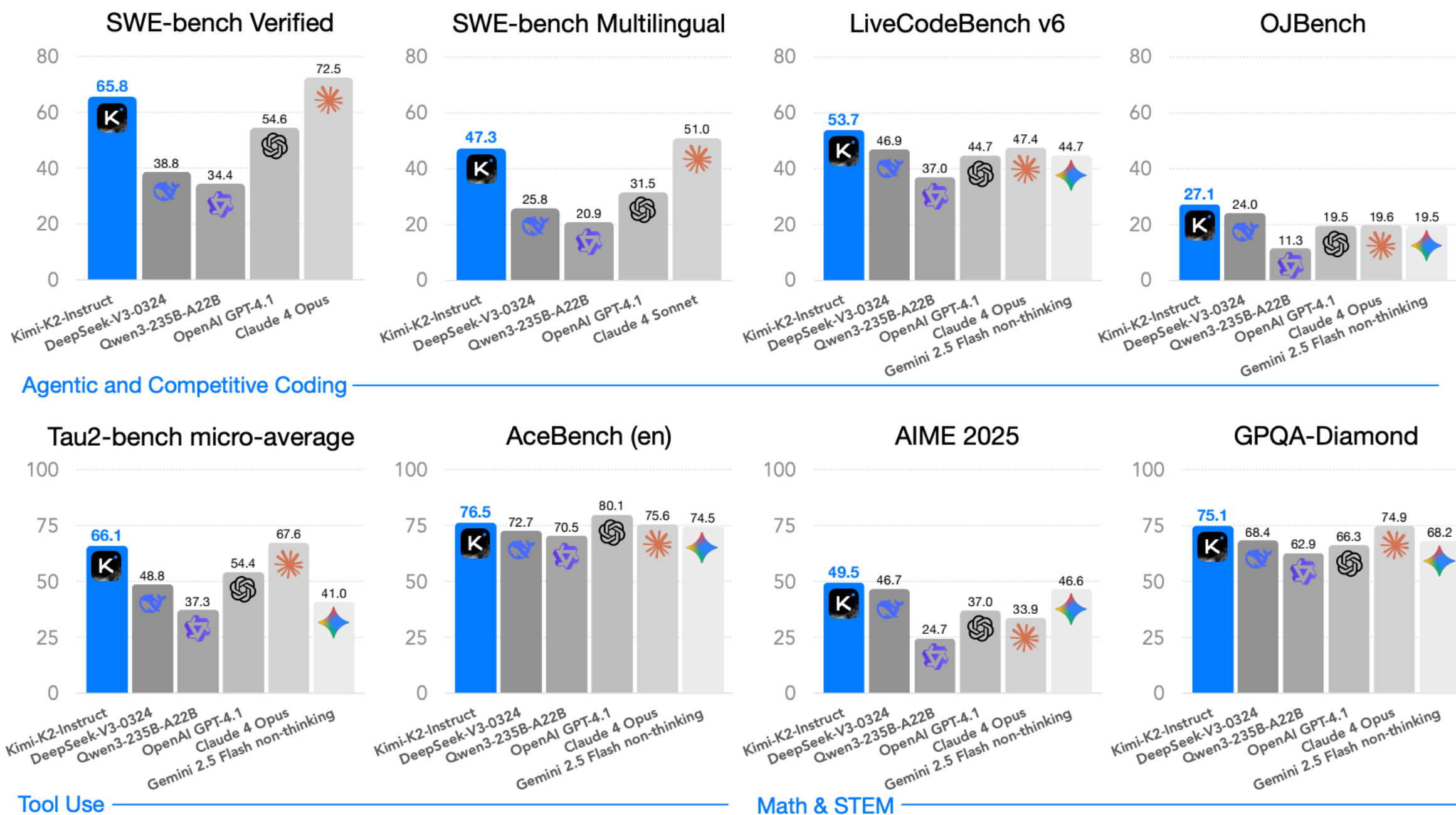


Figure 1: Kimi K2 main results.²

Kimi-K2 SFT

3.1.1 Large-Scale Agentic Data Synthesis for Tool Use Learning

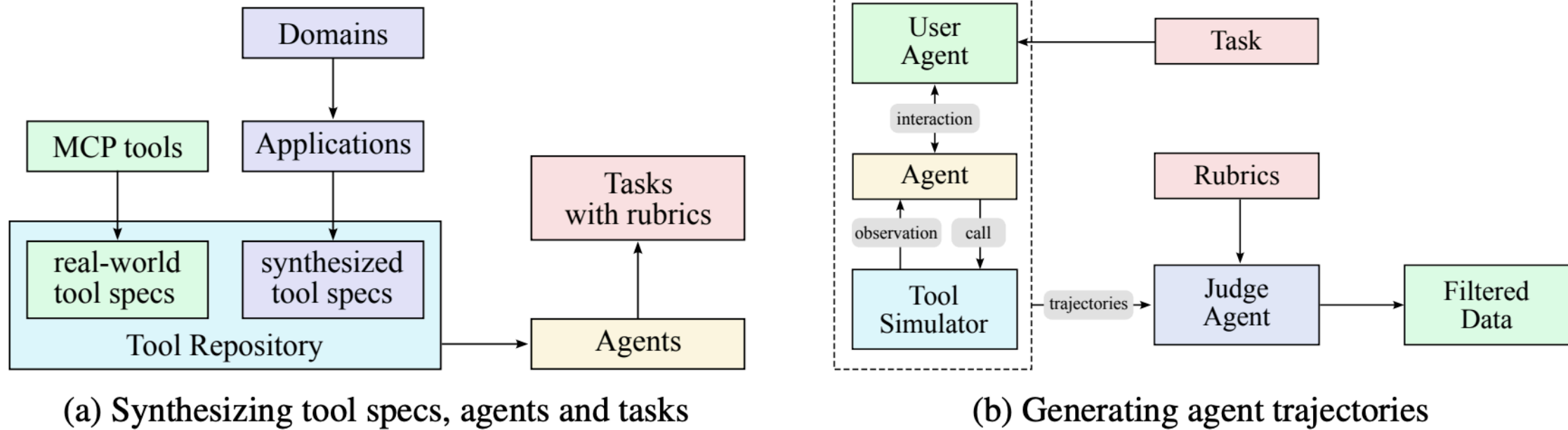


Figure 8: Data synthesis pipeline for tool use. (a) Tool specs are from both real-world tools and LLMs; agents and tasks are the generated from the tool repo. (b) Multi-agent pipeline to generate and filter trajectories with tool calling.

Kimi-K2 RL

3.2.1 Verifiable Rewards Gym

Math, STEM and Logical Tasks For math, stem and logical reasoning domains, our RL data preparation follows two key principles, *diverse coverage* and *moderate difficulty*.

Too easy or too hard questions doesn't benefits RL

3.2.2 Beyond Verification: Self-Critique Rubric Reward

- Generative RMs: Self-critique rubric reward
- Critic model is refined using verifiable signals
 - Distill objective signals into evaluation models

Kimi-K2 GRM Rubrics

F K2 Critic Rubrics for General RL

F.1 Core Rubrics

- **Clarity and Relevance:** Assesses the extent to which the response is succinct while fully addressing the user's intent. The focus is on eliminating unnecessary detail, staying aligned with the central query, and using efficient formats such as brief paragraphs or compact lists. Unless specifically required, long itemizations should be avoided. When a choice is expected, the response should clearly offer a single, well-defined answer.
- **Conversational Fluency and Engagement:** Evaluates the response's contribution to a natural, flowing dialogue that extends beyond simple question-answering. This includes maintaining coherence, showing appropriate engagement with the topic, offering relevant observations or insights, potentially guiding the conversation constructively when appropriate, using follow-up questions judiciously, handling hypothetical or personal-analogy queries gracefully, and adapting tone effectively to suit the conversational context (e.g., empathetic, formal, casual).
- **Objective and Grounded Interaction:** Assesses the response's ability to maintain an objective and grounded tone, focusing squarely on the substance of the user's request. It evaluates the avoidance of both metacommentary (analyzing the query's structure, topic combination, perceived oddity, or the nature of the interaction itself) and unwarranted flattery or excessive praise directed at the user or their input. Excellent responses interact respectfully but neutrally, prioritizing direct, task-focused assistance over commentary on the conversational dynamics or attempts to curry favor through compliments.

F.2 Prescriptive Rubrics

- **Initial Praise:** Responses must not begin with compliments directed at the user or the question (e.g., "That's a beautiful question", "Good question!").
- **Explicit Justification:** Any sentence or clause that explains why the response is good or how it successfully fulfilled the user's request. This is different from simply describing the content.

To eliminate reward hacking

Kimi-K2 RL

3.2.3 RL Algorithm

We adopt the policy optimization algorithm introduced in K1.5 [36] as the foundation for K2. For each problem x , we sample K responses $\{y_1, \dots, y_k\}$ from the previous policy π_{old} , and optimize the model π_{θ} with respect to the following objective:

$$L_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\frac{1}{K} \sum_{i=1}^K \left[\left(r(x, y_i) - \bar{r}(x) - \tau \log \frac{\pi_{\theta}(y_i|x)}{\pi_{\text{old}}(y_i|x)} \right)^2 \right] \right],$$

where $\bar{r}(x) = \frac{1}{k} \sum_{i=1}^k r(x, y_i)$ is the mean rewards of the sampled responses, $\tau > 0$ is a regularization parameter that promotes stable learning. As in SFT, we employ the Muon optimizer [34] to minimize this objective. As we scale RL training to encompass a broader range of tasks in K2, a primary challenge is achieving consistent performance improvements across all domains. To address this, we introduce several additions to the RL algorithm.

$$\pi_{\theta}(y_i \mid x) \propto \pi_{\text{old}}(y_i \mid x) \exp \left(\frac{r(x, y_i) - \bar{r}(x)}{\tau} \right)$$

Kimi-K2 RL

Budget Control It has been widely observed that RL often results in a substantial increase in the length of model-generated responses [36, 20]. While longer responses can enable the model to utilize additional test-time compute for improved performance on complex reasoning tasks, the benefits often do not justify its inference cost in non-reasoning domains. To encourage the model to properly distribute inference budget, we enforce a per-sample *maximum token budget* throughout RL training, where the budget is determined based on the type of task. Responses that exceed this token budget are truncated and assigned a penalty, which incentivizes the model to generate solutions within the specified limit. Empirically, this approach significantly enhances the model’s token efficiency, encouraging concise yet effective solutions across all domains.

PTX Loss To prevent the potential forgetting of valuable, high-quality data during joint RL training, we curate a dataset comprising hand-selected, high-quality samples and integrate it into the RL objective through an auxiliary PTX loss [55]. This strategy not only leverages the advantages of high-quality data, but also mitigates the risk of overfitting to the limited set of tasks explicitly present in the training regime. This augmentation substantially improves the model’s generalization across a broader range of domains.

Temperature Decay For tasks such as creative writing and complex reasoning, we find that promoting exploration via a high sampling temperature during the initial stages of training is crucial. A high temperature allow the model to generate diverse and innovative responses, thereby facilitating the discovery of effective strategies and reducing the risk of premature convergence to suboptimal solutions. However, retaining a high temperature in the later stages of training or during evaluation can be detrimental, as it introduces excessive randomness and compromises the reliability and consistency of the model’s outputs. To address this, we employ a temperature decay schedule, to shift from exploration to exploitation throughout the training. This strategy ensures that the model leverages exploration when it is most beneficial, while ultimately converge on stable and high-quality outputs.

Kimi-K2.5 RL

Policy Optimization For each problem x sampled from a dataset $\mathcal{D}$, K responses $\{y_1, \dots, y_K\}$ are generated using the previous policy π_{old} . We optimize the model π_{θ} with respect to the following objective:

$$L_{\text{RL}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\frac{1}{N} \sum_{j=1}^K \sum_{i=1}^{|y_j|} \text{Clip} \left(\frac{\pi_{\theta}(y_j^i | x, y_j^{0:i})}{\pi_{\text{old}}(y_j^i | x, y_j^{0:i})}, \alpha, \beta \right) (r(x, y_j) - \bar{r}(x)) - \tau \left(\log \frac{\pi_{\theta}(y_j^i | x, y_j^{0:i})}{\pi_{\text{old}}(y_j^i | x, y_j^{0:i})} \right)^2 \right]. \quad (1)$$

Here $\alpha, \beta, \tau > 0$ are hyperparameters, $y_{0:i}^j$ is the prefix up to the i -th token of the j -th response, $N = \sum_{i=1}^K |y_i|$ is the total number of generated tokens in a batch, $\bar{r}(x) = \frac{1}{K} \sum_{j=1}^K r(x, y_j)$ is the mean reward of all generated responses.

Verifiable rewards + Generative Reward Models

Kimi-K2.5 Token-Efficient RL

To this end, we propose *Toggle*, a training heuristic that alternates between inference-time scaling and budget-constrained optimization: for learning iteration t , the reward function is defined by

$$\tilde{r}(x, y) = \begin{cases} r(x, y) \cdot \mathbb{I} \left\{ \frac{1}{K} \sum_{i=1}^K r(x, y_i) < \lambda \text{ or } |y_i| \leq \text{budget}(x) \right\} & \text{if } \lfloor t/m \rfloor \pmod{2} = 0 \text{ (Phase0)} \\ r(x, y) & \text{if } \lfloor t/m \rfloor \pmod{2} = 1 \text{ (Phase1)} \end{cases} .$$

- *Phase0 (budget limited phase)*: The model is trained to solve the problem within a task-dependent token budget. To prevent a premature sacrifice of quality for efficiency, this constraint is conditionally applied: it is only enforced when the model’s mean accuracy for a given problem exceeds the threshold λ .
- *Phase1 (standard scaling phase)*: The model generates responses up to the maximum token limit, encouraging the model to leverage computation for better inference-time scaling.

The problem-dependent budget is estimated from the ρ -th percentile of token lengths among the subset of correct responses:

$$\text{budget}(x) = \text{Percentile}(\{|y_j| \mid r(x, y_i) = 1, i = 1, \dots, K\}, \rho) . \quad (2)$$

GLM-5: from Vibe Coding to Agentic Engineering

GLM-5 Team

Zhipu AI & Tsinghua University

(For the complete list of authors, please refer to the Contribution section)

Abstract

We present GLM-5, a next-generation foundation model designed to transition the paradigm of vibe coding to agentic engineering. Building upon the agentic, reasoning, and coding (ARC) capabilities of its predecessor, GLM-5 adopts DSA to significantly reduce training and inference costs while maintaining long-context fidelity. To advance model alignment and autonomy, we implement a new asynchronous reinforcement learning infrastructure that drastically improves post-training efficiency by decoupling generation from training. Furthermore, we propose novel asynchronous agent RL algorithms that further improve RL quality, enabling the model to learn from complex, long-horizon interactions more effectively. Through these innovations, GLM-5 achieves state-of-the-art performance on major open benchmarks. Most critically, GLM-5 demonstrates unprecedented capability in real-world coding tasks, surpassing previous baselines in handling end-to-end software engineering challenges. Code, models, and more information are available at <https://github.com/zai-org/GLM-5>.

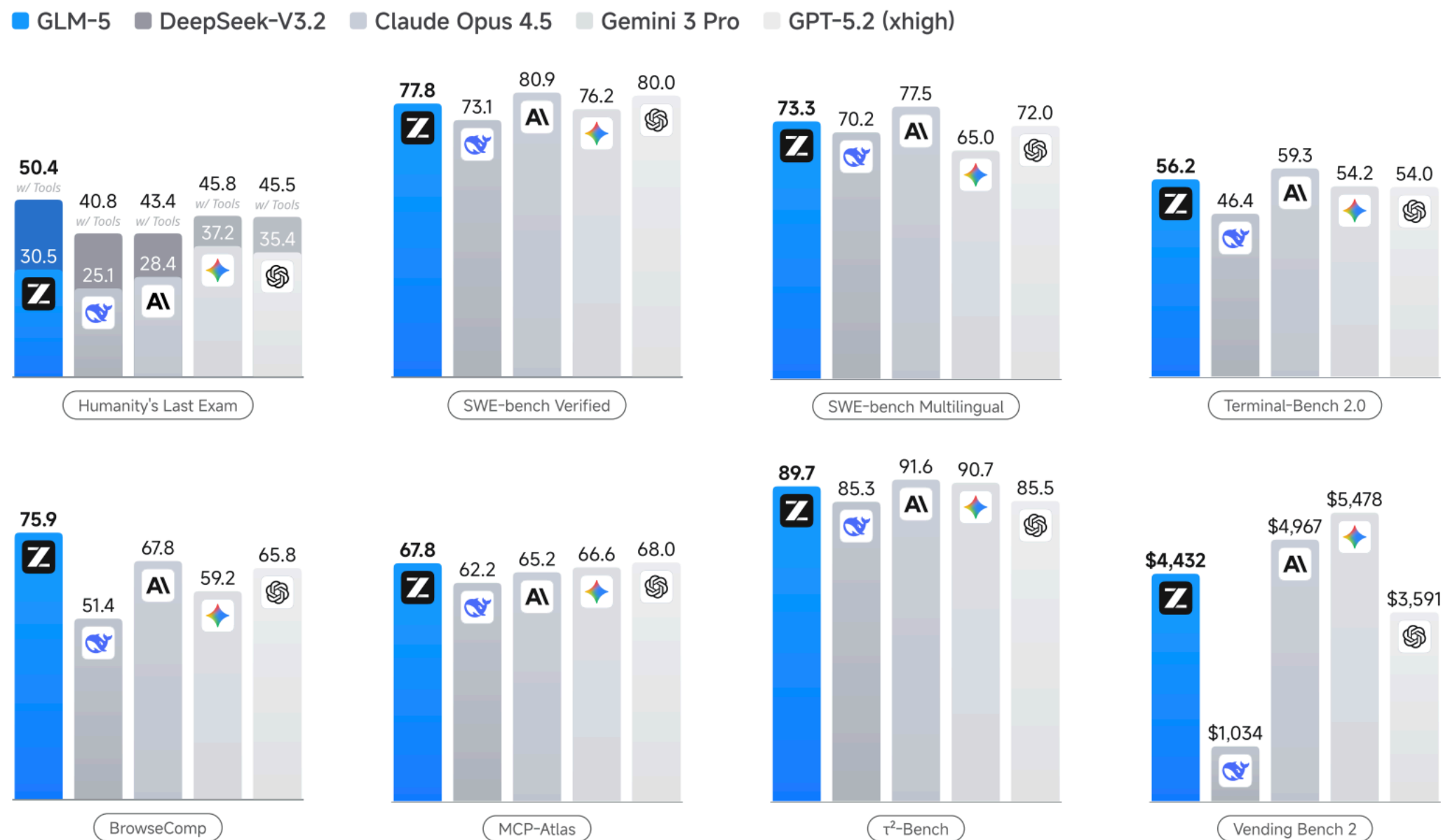


Figure 1: Results of GLM-5, DeepSeek-V3.2, Claude Opus 4.5, Gemini 3 Pro, and GPT-5.2 (xhigh) on 8 agentic, reasoning, and coding benchmarks: Humanity’s Last Exam, SWE-bench Verified, SWE-bench Multilingual, Terminal-Bench 2.0, BrowseComp, MCP-Atlas, τ^2 -Bench, Vending Bench 2.

GLM-5

3.1 Supervised Fine-Tuning

Compared with GLM-4.5, GLM-5 significantly expands the scale of *Agent* and *Coding* data during the SFT stage. The SFT corpus of GLM-5 covers three major categories:

- **General Chat:** question answering, writing, role-playing, translation, multi-turn dialogue, and long-context interactions;
- **Reasoning:** mathematical, programming, and scientific reasoning;
- **Coding & Agent:** frontend and backend engineering code, tool calling, coding agents, search agents, and general-purpose agents.

Long-context SFT

- **Interleaved Thinking:** the model thinks before every response and tool call, improving instruction following and the quality of generation⁴.
- **Preserved Thinking:** in coding agent scenarios, the model automatically retains all thinking blocks across multi-turn conversations, reusing existing reasoning instead of re-deriving it from scratch. This reduces information loss and inconsistencies, and is well-suited for long-horizon, complex tasks⁵.
- **Turn-level Thinking:** the model supports per-turn control over reasoning within a session—disable thinking for lightweight requests to reduce latency/cost, enable it for complex tasks to improve accuracy and stability.

GLM-5

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}^{\text{infer}}(\cdot | x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \text{pop}(\rho_{i,t}, 1/\beta, \beta) \cdot \min \left(r_{i,t} \hat{A}_{i,t}, \text{clip}(r_{i,t}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \hat{A}_{i,t} \right) \right], \quad (1)$$

where the training-inference mismatch ratio is defined as

$$\rho_{i,t} = \frac{\pi_{\theta_{\text{old}}}^{\text{train}}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}^{\text{infer}}(y_{i,t} \mid x, y_{i,<t})}.$$

The operator $\text{pop}(\cdot)$ suppresses samples whose mismatch ratio deviates excessively:

$$\text{pop}(\rho_{i,t}, 1/\beta, \beta) = \begin{cases} \rho_{i,t}, & 1/\beta \leq \rho_{i,t} \leq \beta, \\ 0, & \text{otherwise.} \end{cases}$$



Small Leak Can Sink a Great Ship— Boost RL Training on MoE with *IcePop*!

Xin Zhao*, Yongkang Liu*, Kuan Xu*, Jia Guo*, Zihao Wang^{*†}, Yan Sun, Xinyu Kong,
Qianggang Cao, Liang Jiang, Zujie Wen^{*†}, Zhiqiang Zhang, Jun Zhou

* Core Contributors, [†] Project Leads.

ICEPOP

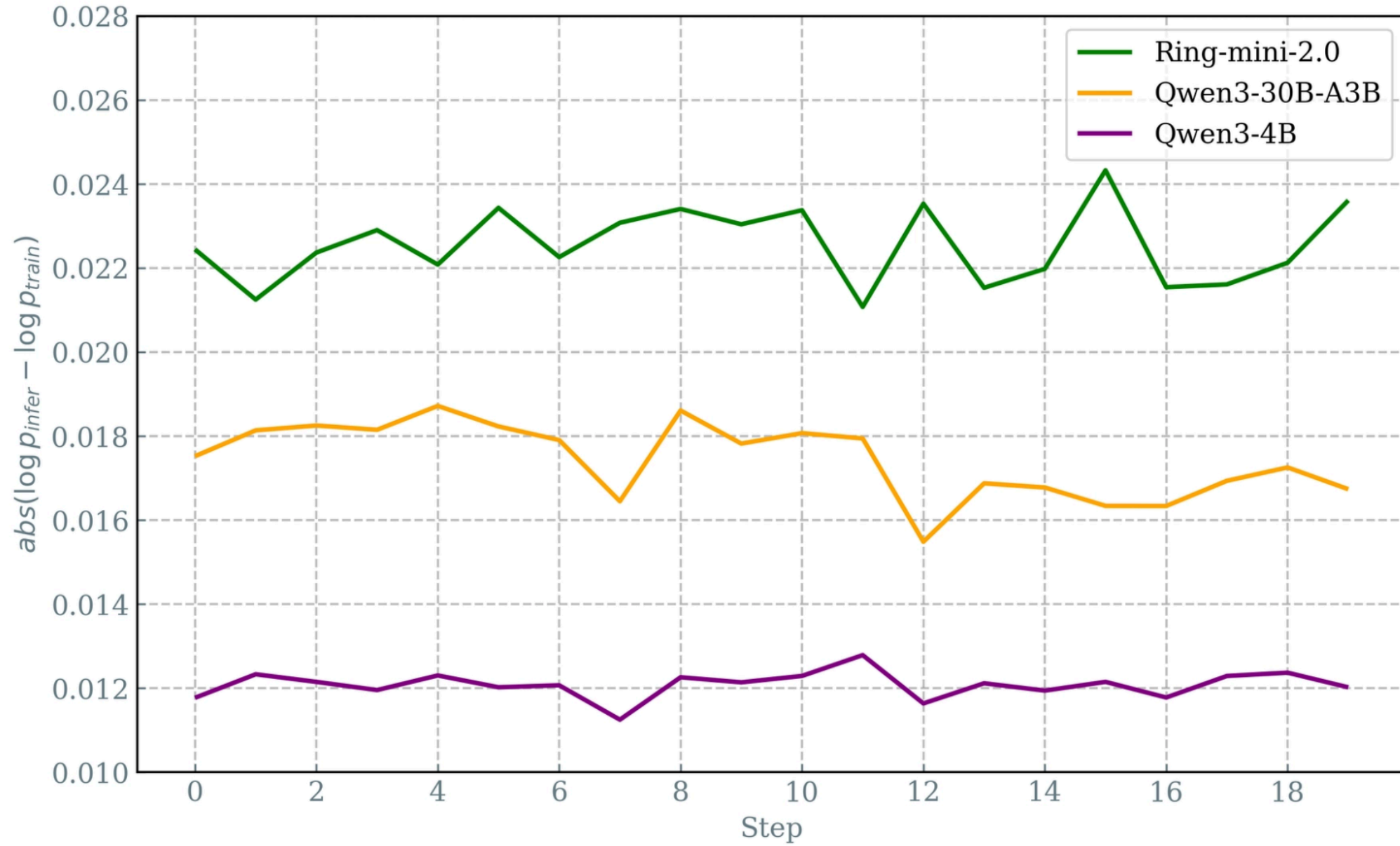


Figure 1. The comparisons of $|\log p_{infer} - \log p_{train}|$ between MoE and Dense models. We select three representatives: Ring-mini-2.0 (MoE), Qwen3-4B (Dense), Qwen3-30B-A3B (MoE), which shows that the MoE models usually exhibit greater discrepancy between training and inference engines. We will include more comparisons across varied model sizes.

ICEPOP

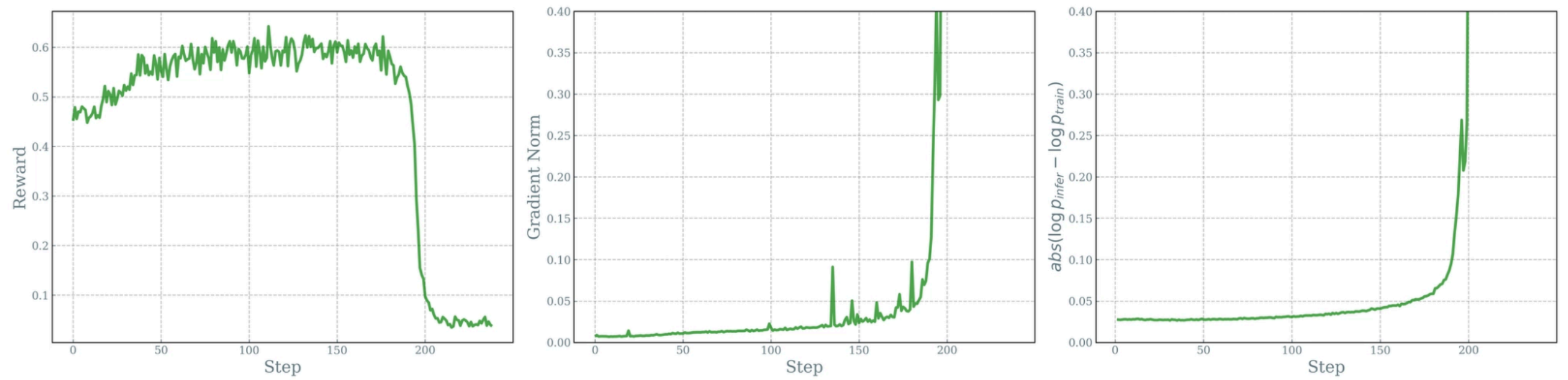


Figure 4. *The training signals from the crashed model.*

ICEPOP

$$\mathcal{J}_{\text{IcePop}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\text{infer}}(\cdot | x; \theta_{\text{old}})} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \left[\mathcal{M} \left(\frac{\pi_{\text{train}}(y_{i,t} | x, y_{i,<t}; \theta_{\text{old}})}{\pi_{\text{infer}}(y_{i,t} | x, y_{i,<t}; \theta_{\text{old}})}; \alpha, \beta \right) \right. \right. \\ \left. \left. \cdot \min \left(r_{i,t} \hat{A}_{i,t}, \text{clip} \left(r_{i,t}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right) \right] \right]$$

where $r_{i,t} = \frac{\pi_{\text{train}}(y_{i,t} | x, y_{i,<t}; \theta)}{\pi_{\text{train}}(y_{i,t} | x, y_{i,<t}; \theta_{\text{old}})}$, with the masking function below,

$$\mathcal{M}(k) = \begin{cases} k & \text{if } k \in [\alpha, \beta] \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

and two hyperparameters α, β to control the lower and upper limits.

ICEPOP

- **Model Evaluation:** On the challenging benchmark AIME25, **IcePop** consistently outperforms **TIS** with a large gain along the training process, and finally improves the base score (63%) by over 14%, and expands the performance gap with **TIS** by relative 6%.

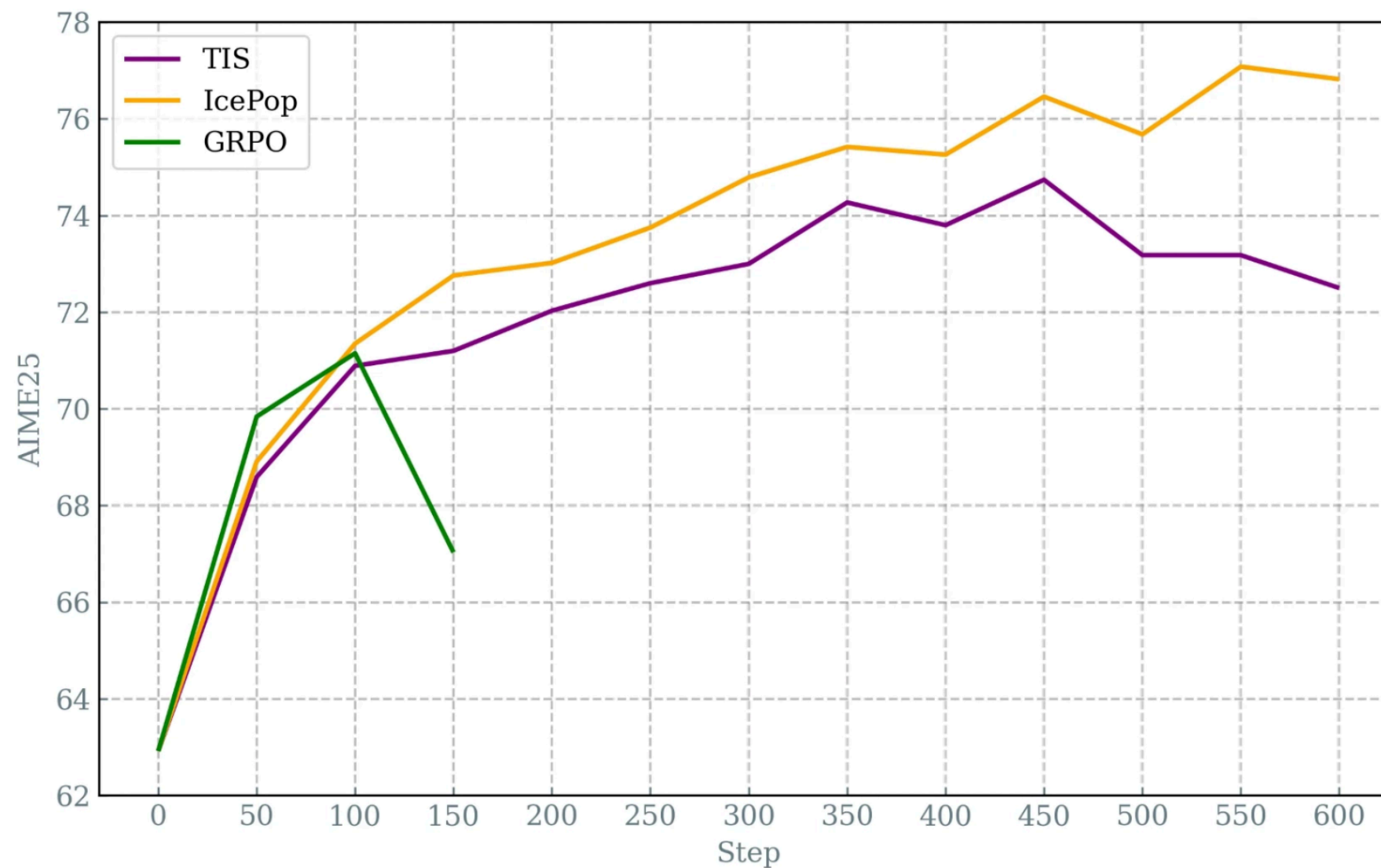


Figure 5. The performance comparisons of Avg@64 on AIME25. We evaluate all models using the same setting.

ICEPOP

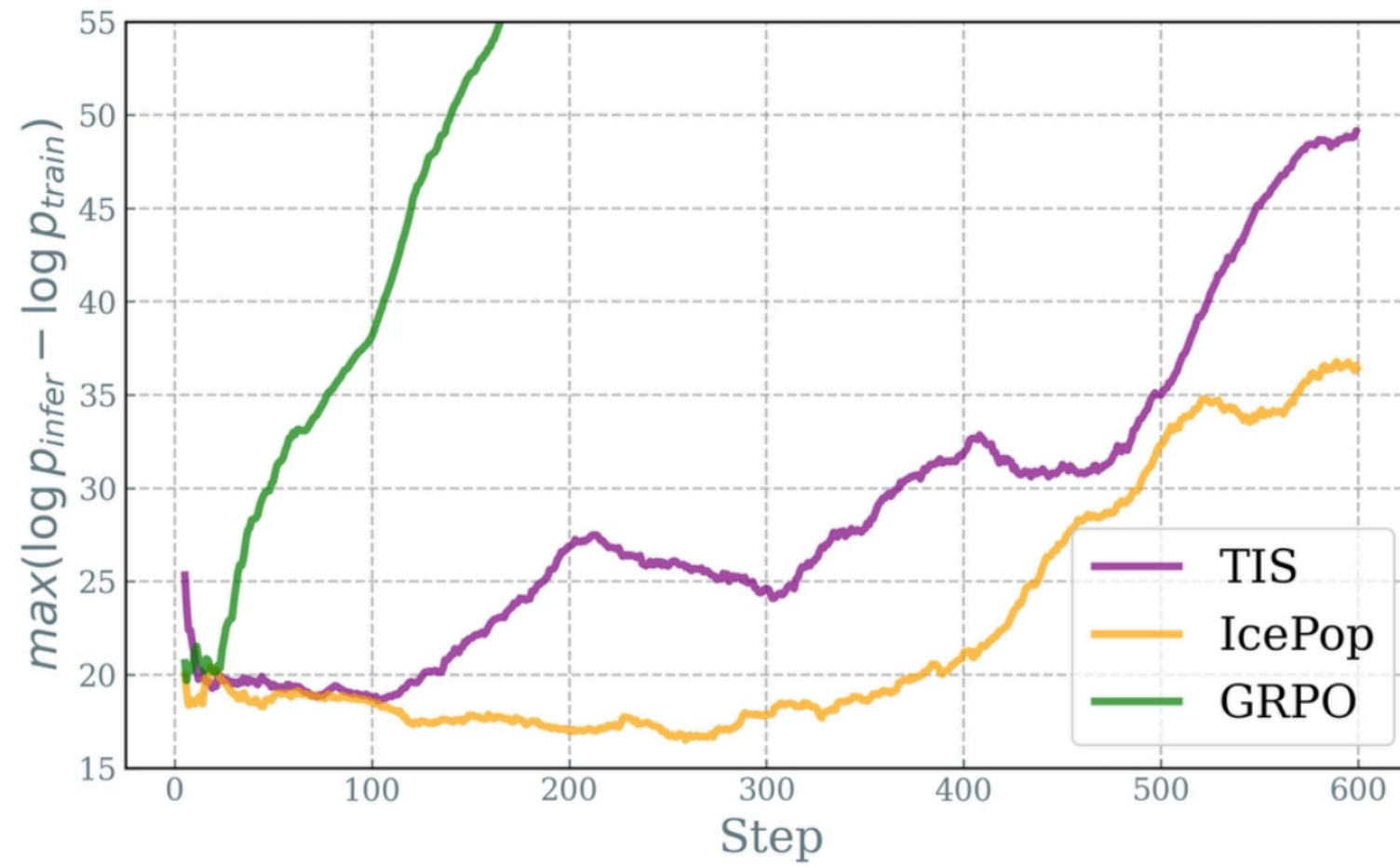


Figure 6. The maximum of probability discrepancy.

ICEPOP

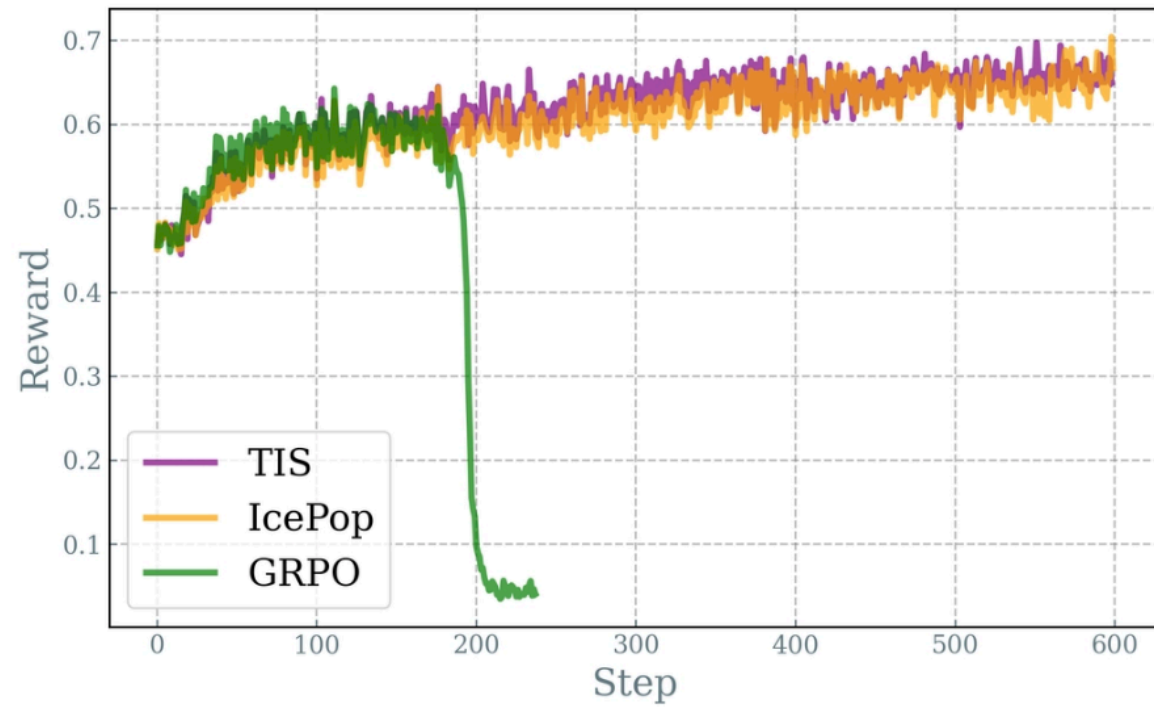


Figure 7. Training reward. The reward of baseline collapses after 180–200 steps. Both IcePop and TIS maintain stable growth.

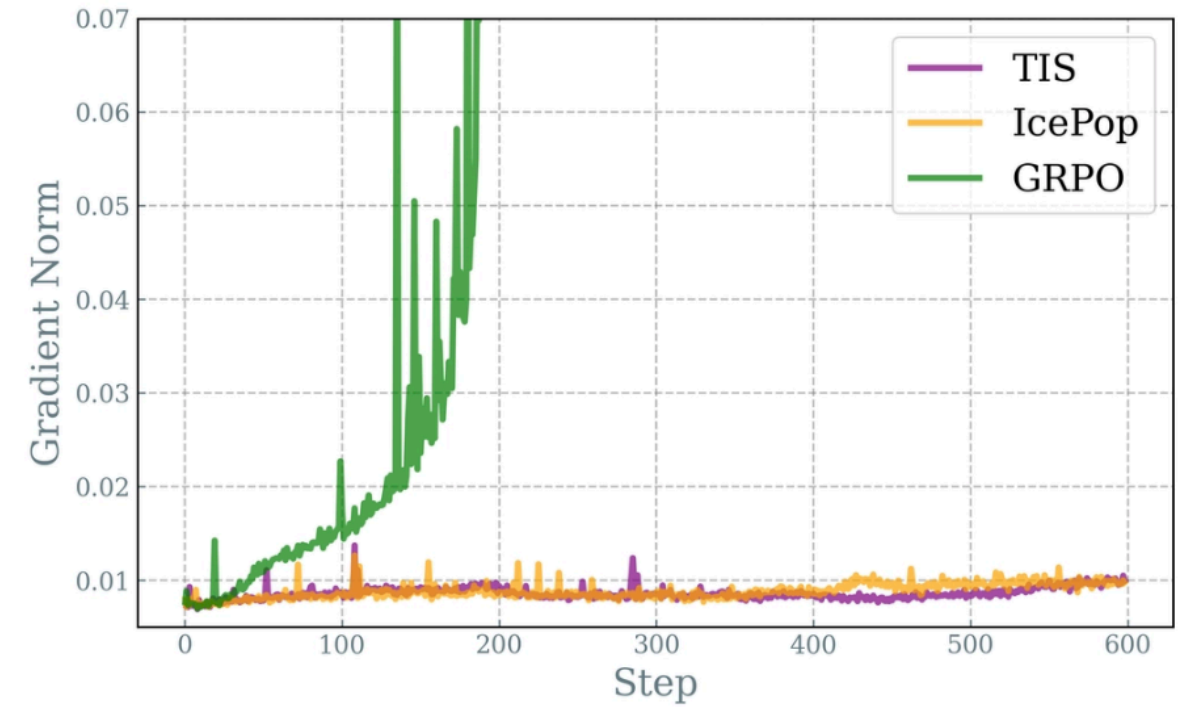


Figure 8. Gradient norm. Baseline explodes, IcePop and TIS remain stable.

GLM-5

- Mixed domain reasoning RL
 - Math, sciences, code, tool-use, etc.
- Multi-dimensional optimization objectives:
 - Correctness, emotional intelligence, task-specific qualities
- Hybrid reward system
 - Rule-based rewards, outcome rewards, generative RMs
- Human-in-the-loop alignment
 - Rather than relying solely on model-generated responses, they introduce expert human responses as stylistic and qualitative anchors
- On-policy distillation
 - Merge capabilities across different SFT and RL ckpts
 - Batch size 1024, group size 1

Break

MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention

MiniMax¹

We introduce MiniMax-M1, the world’s first open-weight, large-scale hybrid-attention reasoning model. MiniMax-M1 is powered by a hybrid Mixture-of-Experts (MoE) architecture combined with a lightning attention mechanism. The model is developed based on our previous MiniMax-Text-01 model (MiniMax et al., 2025), which contains a total of 456 billion parameters with 45.9 billion parameters activated per token. The M1 model natively supports a context length of 1 million tokens, 8x the context size of DeepSeek R1. Furthermore, the lightning attention mechanism in MiniMax-M1 enables efficient scaling of test-time compute – For example, compared to DeepSeek R1, M1 consumes 25% of the FLOPs at a generation length of 100K tokens. These properties make M1 particularly suitable for complex tasks that require processing long inputs and thinking extensively. MiniMax-M1 is trained using large-scale reinforcement learning (RL) on diverse problems ranging from traditional mathematical reasoning to sandbox-based, real-world software engineering environments. In addition to the inherent efficiency advantage of lightning attention for RL training, we propose CISPO, a novel RL algorithm to further enhance RL efficiency. CISPO clips importance sampling weights rather than token updates, outperforming other competitive RL variants. Combining hybrid-attention and CISPO enables MiniMax-M1’s full RL training on 512 H800 GPUs to complete in only three weeks, with a rental cost of just \$534,700. We release two versions of MiniMax-M1 models with 40K and 80K thinking budgets respectively, where the 40K model represents an intermediate phase of the 80K training. Experiments on standard benchmarks show that our models are comparable or superior to strong open-weight models such as the original DeepSeek-R1 and Qwen3-235B, with particular strengths in complex software engineering, tool utilization, and long-context tasks. Through efficient scaling of test-time compute, MiniMax-M1 serves as a strong foundation for next-generation language model agents to reason and tackle real-world challenges. We publicly release MiniMax-M1 at <https://github.com/MiniMax-AI/MiniMax-M1>.

MiniMax-M1

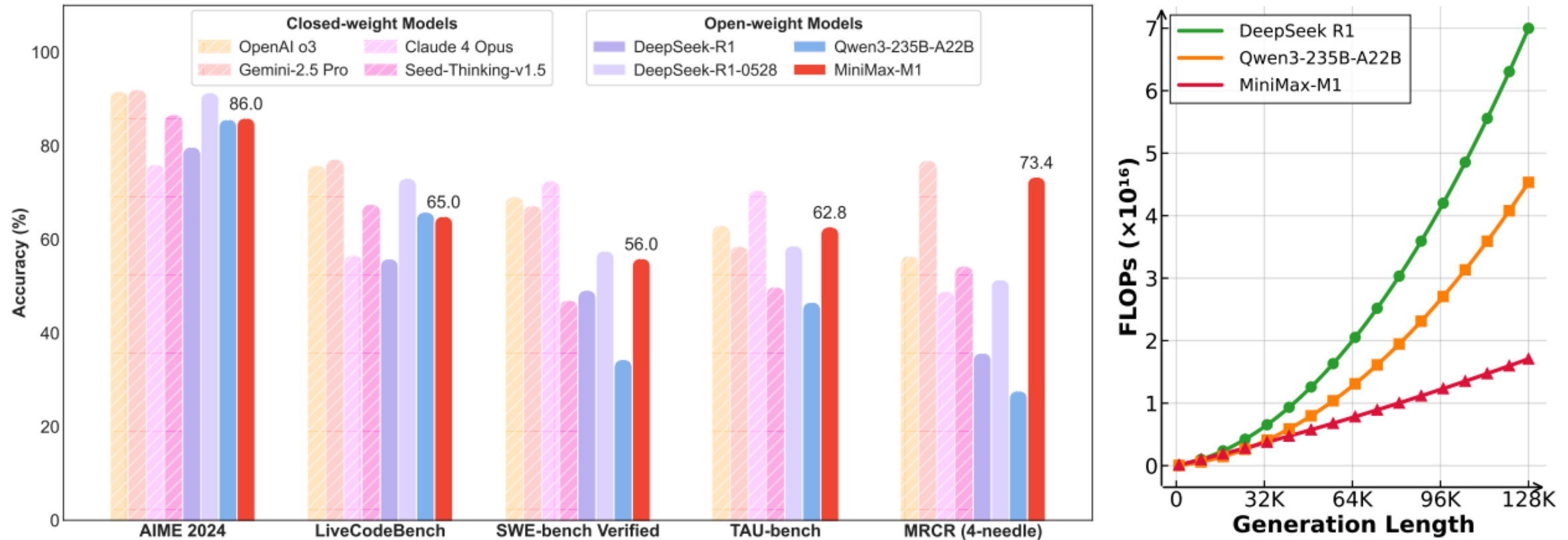


Figure 1 | **Left:** Benchmark performance comparison of leading commercial and open-weight models across competition-level mathematics, coding, software engineering, agentic tool use, and long-context understanding tasks. We use the MiniMax-M1-80k model here for MiniMax-M1. **Right:** Theoretical inference FLOPs scaling with generation length (# tokens).

MiniMax-M1

- Continual Pre-training
 - Prioritize question-answer pairs
 - Increase STEM, code, and reasoning-based data to 70%
- SFT
 - Elicit desired behaviors such as reflection-based CoT
 - Use data samples with long CoT
 - 60% data are math and coding

Prepare a good initialization for RL;

If desired behaviors are unlikely to appear, we can waste RL compute.

MiniMax-M1

$$\mathcal{J}_{\text{PPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, o_i \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) - \beta D_{KL}(\pi_{\theta} || \pi_{\text{ref}}) \right], \quad (1)$$

where $r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})}$ is the importance sampling (IS) weight, which is used to correct the

- In Zero-RL setting
 - Reflection tokens (e.g., however, wait) have low-prob
 - If we do multiple mini-batch updates per rollout
 - Later updates will likely got clipped

MiniMax-M1

our approach CISPO (**C**lipped **I**S-weight **P**olicy **O**ptimization). Adopting the group relative advantage from GRPO and the token-level loss (Liu et al., 2025b; Yu et al., 2025), CISPO optimizes the following objective:

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) \right], \quad (4)$$

where $\hat{r}_{i,t}(\theta)$ is the clipped IS weight:

$$\hat{r}_{i,t}(\theta) = \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}^{\text{IS}}, 1 + \epsilon_{\text{high}}^{\text{IS}} \right). \quad (5)$$

MiniMax-M1

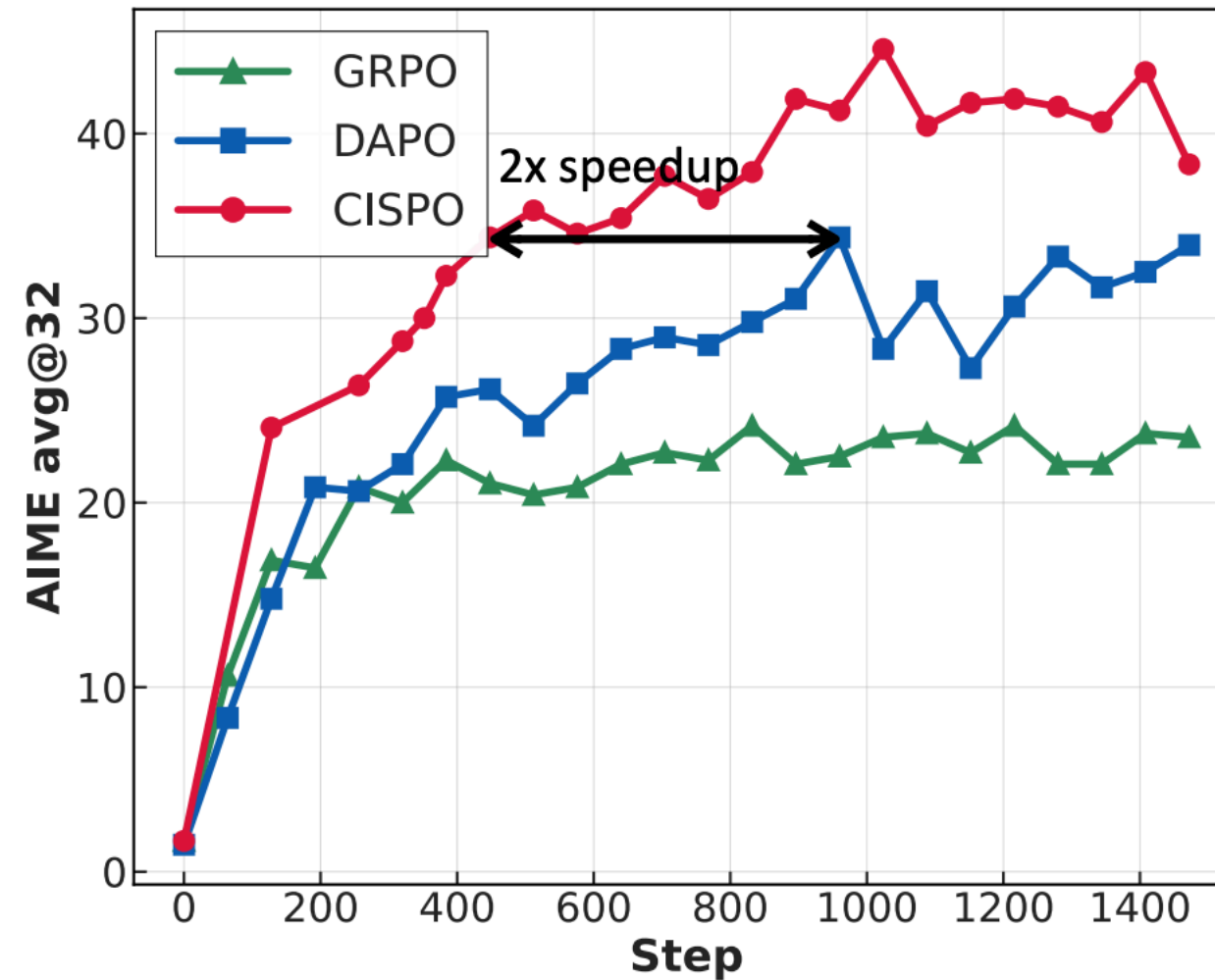


Figure 2 | Comparison of GRPO, DAPO, and our proposed CISPO on AIME 2024, based on Qwen2.5-32B-base. CISPO outperforms both GRPO and DAPO in terms of performance at the same number of training steps, and achieves comparable performance to DAPO using 50% of the training steps.

MiniMax-M1

A General Formulation. While we adopt CISPO in our experiments, here we further present a unified formulation by introducing a token-wise mask into the CISPO objective. This allows for hyperparameter tuning to control whether, and under what conditions, gradients from specific tokens should be dropped:

$$\mathcal{J}_{\text{unify}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) M_{i,t} \right]. \quad (6)$$

The mask $M_{i,t}$ is equivalent to the mask implicitly defined in the PPO trust region:

$$M_{i,t} = \begin{cases} 0 & \text{if } \hat{A}_{i,t} > 0 \text{ and } r_{i,t}(\theta) > 1 + \epsilon_{\text{high}}, \\ 0 & \text{if } \hat{A}_{i,t} < 0 \text{ and } r_{i,t}(\theta) < 1 - \epsilon_{\text{low}}, \\ 1 & \text{otherwise.} \end{cases} \quad (7)$$

This unified loss formulation can flexibly represent different clipping strategies under a common framework.

MiniMax-M1

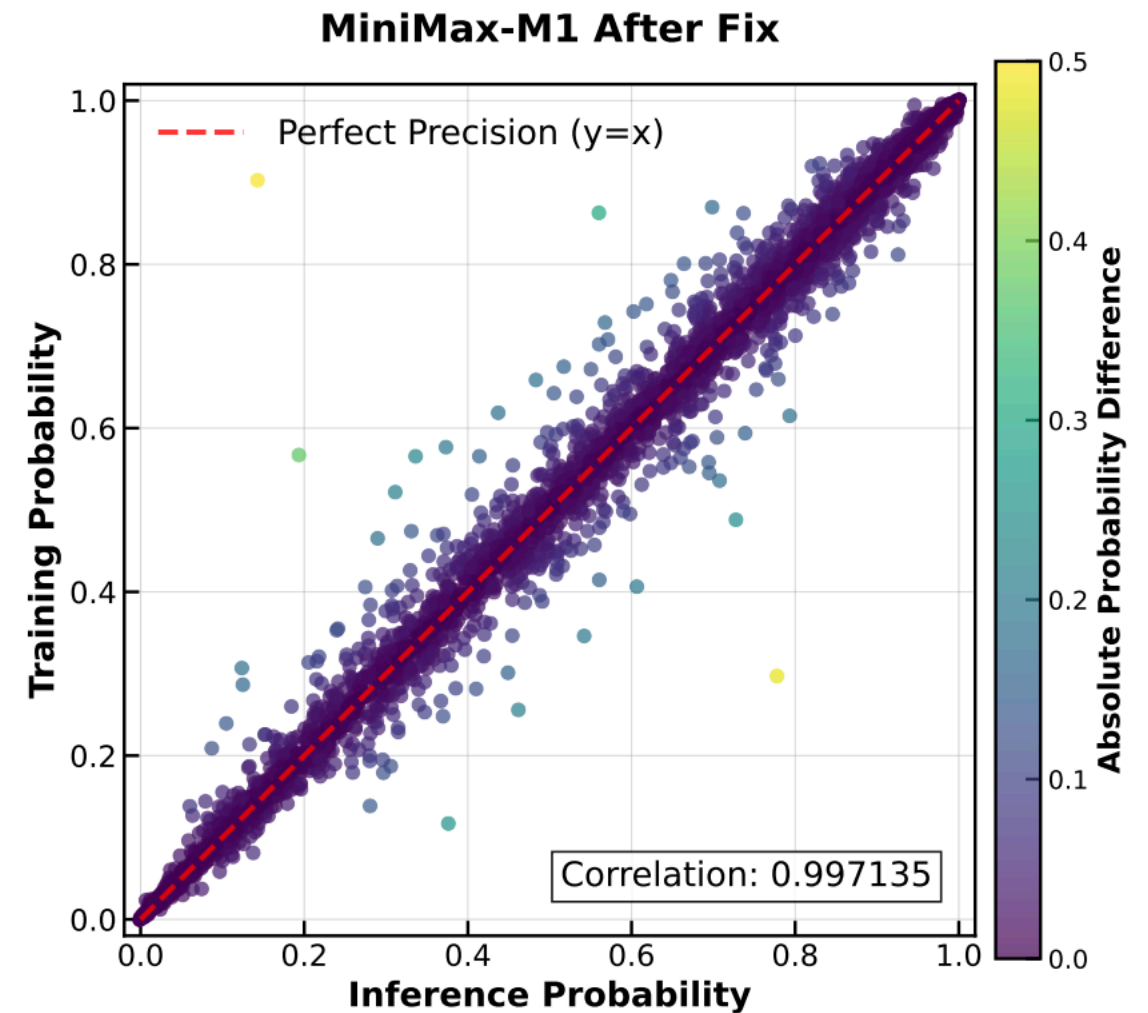
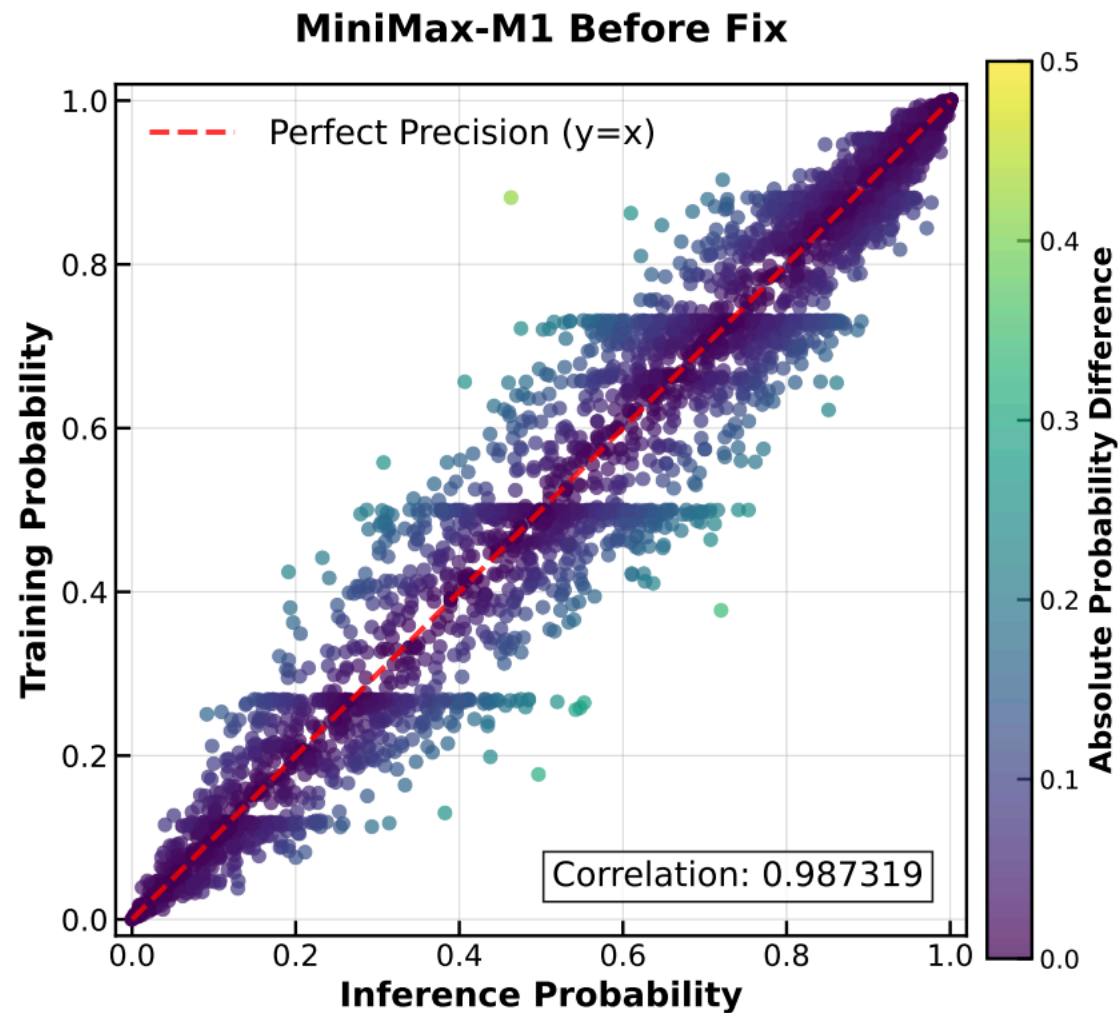


Figure 3 | Probability of tokens in training-mode code vs. probability of tokens in inference-mode code. Each point in the figures represents an individual token. The Pearson correlation coefficient is indicated in the figures. Theoretically, the two probabilities should be identical, and all the tokens should be exactly on the diagonal line. **Left:** Correlation of the M1 model before our fix; **Right:** Correlation of the M1 model after applying our fix of using FP32 precision for the LM output head.

Deal with differences between rollouts and training

MiniMax-M1

- Optimizer hypparameters
 - Adam betas (0.9, 0.999) $\rightarrow$ (0.9, 0.95)
 - eps $1e-8 \rightarrow 1e-15$: grad magnitudes span $1e-18 - 1e-5$
- Early truncation via repetition detection
 - Early truncation if 3000 consecutive tokens each have probability > 0.99

MiniMax-M1

- RL data
 - Data w/ rule-based verification:
 - Math - 50k samples; logical reasoning - 53k samples; competitive programming - 30k samples; software engineering - several thousands
 - General domain - 25k in total
 - Data from STEM domains, instruction-following data, creative writing, etc.
 - Use generative RMs to score

MiniMax-M1

4.3. Curriculum of Incorporating Diverse Data

Given that our RL data spans a wide spectrum of categories, a core challenge is training a single policy capable of excelling on both reasoning-intensive tasks and general domain tasks. To address this, our approach entails a carefully managed curriculum and dynamic weighting strategy for reasoning and general-domain tasks during the RL training process with CISPO: we start with only the reasoning-intensive tasks with rule-based reward, and then gradually mix in the general domain tasks. This ensures that the model continues to refine its verifiable skills (e.g., in math and code) while progressively enhancing its performance on a diverse spectrum of general tasks, from complex instruction following to open-ended CoT reasoning. This mixed RL training encourages the model to learn context-dependent application of its reasoning abilities—applying rigorous, step-by-step deduction for verifiable problems and more flexible, adaptive generation for general queries—all within a unified policy framework. It prevents catastrophic forgetting of specialized skills while fostering broader generalization.

The Art of Scaling Reinforcement Learning Compute for LLMs

Devvrit Khatri^{2,*†}, Lovish Madaan^{1,3,*},
Rishabh Tiwari^{4†}, Rachit Bansal^{5†}, Sai Surya Duvvuri^{2†}, Manzil Zaheer^{1†},
Inderjit S. Dhillon², David Brandfonbrener¹, Rishabh Agarwal^{6,†}

¹Meta, ²UT Austin, ³UCL, ⁴UC Berkeley, ⁵Harvard University, ⁶Periodic Labs

*Equal contribution, †Work done at Meta

Reinforcement learning (RL) has become central to training large language models (LLMs), yet the field lacks predictive scaling methodologies comparable to those established for pre-training. Despite rapidly rising compute budgets, there is no principled understanding of how to evaluate algorithmic improvements for scaling RL compute. We present the first large-scale systematic study, amounting to more than 400,000 GPU-hours, that defines a principled framework for analyzing and predicting RL scaling in LLMs. We fit sigmoidal compute-performance curves for RL training and ablate a wide range of common design choices to analyze their effects on asymptotic performance and compute efficiency. We observe: (1) Not all recipes yield similar asymptotic performance, (2) Details such as loss aggregation, normalization, curriculum, and off-policy algorithm primarily modulate compute efficiency without materially shifting the asymptote, and (3) Stable, scalable recipes follow predictable scaling trajectories, enabling extrapolation from smaller-scale runs. Combining these insights, we propose a *best-practice* recipe, **ScaleRL**, and demonstrate its effectiveness by successfully scaling and predicting validation performance on a single RL run scaled up to 100,000 GPU-hours. Our work provides both a *scientific framework* for analyzing scaling in RL and a practical recipe that brings RL training closer to the predictability long achieved in pre-training.

Correspondence: {lovish, brandfon}@meta.com, {devvrit.03, rishabhagarwal.467}@gmail.com



ScaleRL

$$\underbrace{R_C - R_0}_{\text{Reward Gain}} = \underbrace{(A - R_0)}_{\text{Asymptotic Reward Gain}} \times \underbrace{\frac{1}{1 + (C_{\text{mid}}/C)^B}}_{\text{Compute Efficiency}} \quad (\text{fixed model and training data}) \quad (1)$$

where $0 \leq A \leq 1$ represents the asymptotic pass rate, $B > 0$ is a scaling exponent that determines the compute efficiency, and C_{mid} sets the midpoint of the RL performance curve. A schematic interpretation of these parameters is provided in [Figure 3](#).

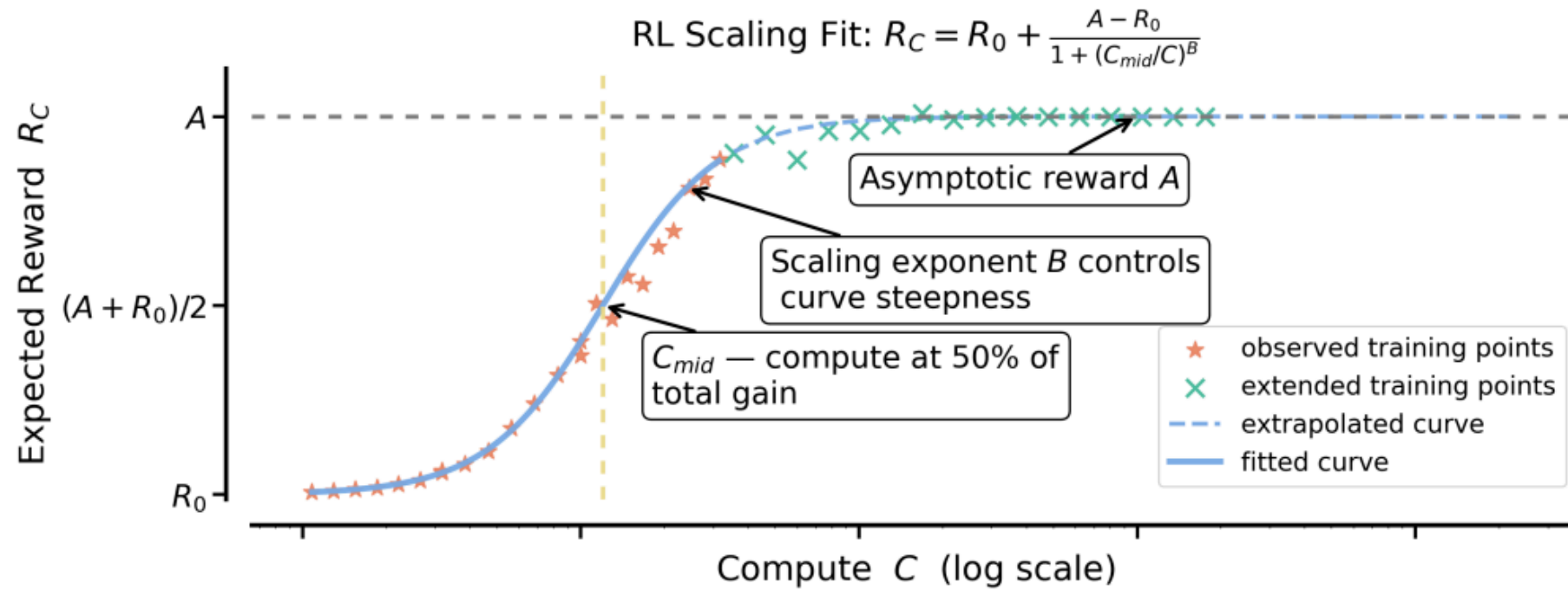


Figure 3 Interpreting [equation \(1\)](#). We provide an example fit illustrating the roles of parameters A , B , and C_{mid} . C_{mid} determines the compute point at which half of the total gain is achieved - smaller values correspond to faster ascent toward the asymptote. B controls the curve’s steepness, with larger values indicating greater efficiency. A represents the asymptotic performance reached at large compute scales. Further discussion is provided in [Appendix A.8](#).

ScaleRL

$$\underbrace{R_C - R_0}_{\text{Reward Gain}} = \underbrace{(A - R_0)}_{\text{Asymptotic Reward Gain}} \times \underbrace{\frac{1}{1 + (C_{\text{mid}}/C)^B}}_{\text{Compute Efficiency}} \quad (\text{fixed model and training data}) \quad (1)$$

where $0 \leq A \leq 1$ represents the asymptotic pass rate, $B > 0$ is a scaling exponent that determines the compute efficiency, and C_{mid} sets the midpoint of the RL performance curve. A schematic interpretation of these parameters is provided in Figure 3.

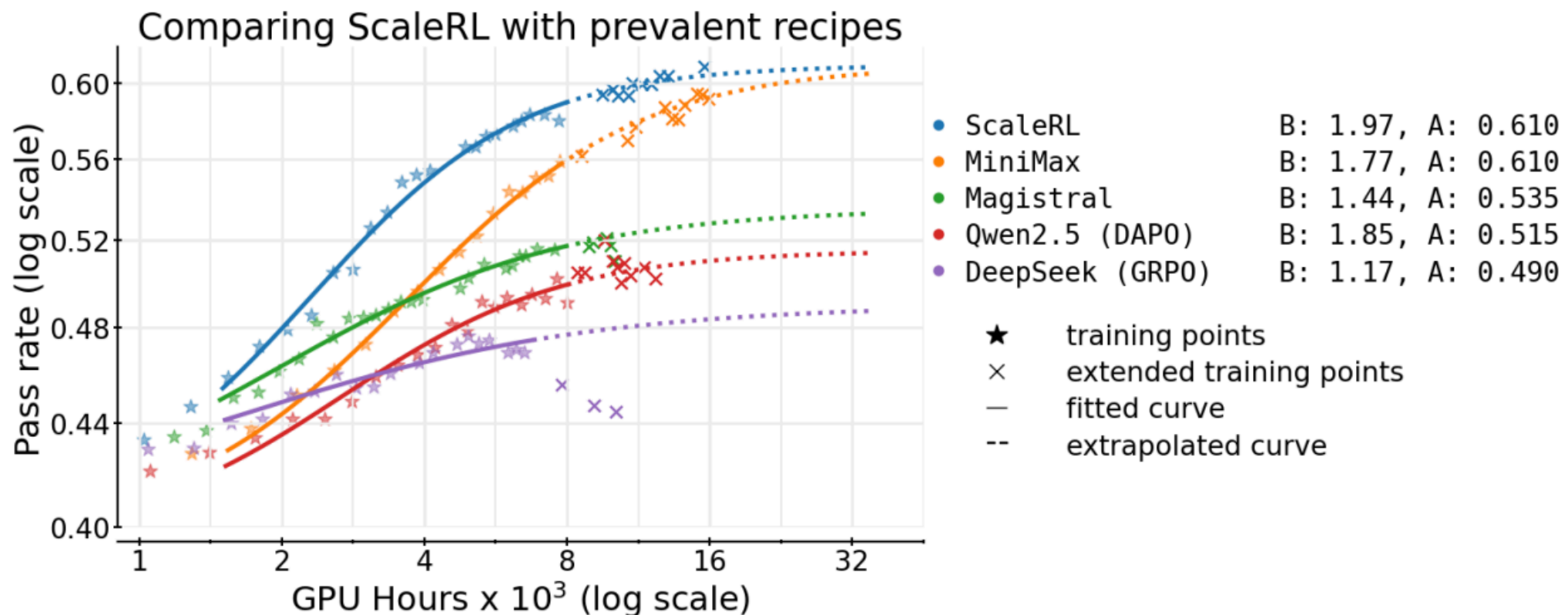


Figure 2 ScaleRL is more scalable than prevalent RL methods. We fit sigmoid curves (Equation 1) on *iid* validation dataset to commonly-used training recipes like DeepSeek (GRPO) (Guo et al., 2025), Qwen-2.5 (DAPO) (Yu et al., 2025), Magistral (Rastogi et al., 2025), and Minimax-M1 (MiniMax et al., 2025), and compare them with **ScaleRL**. **ScaleRL** surpasses all other methods, achieving an asymptotic reward of $A = 0.61$. Stars denote evaluation points; solid curves show the fitted curve over the range used for fitting; dashed curves extrapolate beyond it. We validate the predictability by running each method for longer (“×” markers), which align closely with the extrapolated curves for stable recipes like **ScaleRL** and MiniMax. Further description of the individual recipes compared are given in Appendix A.16.

ScaleRL

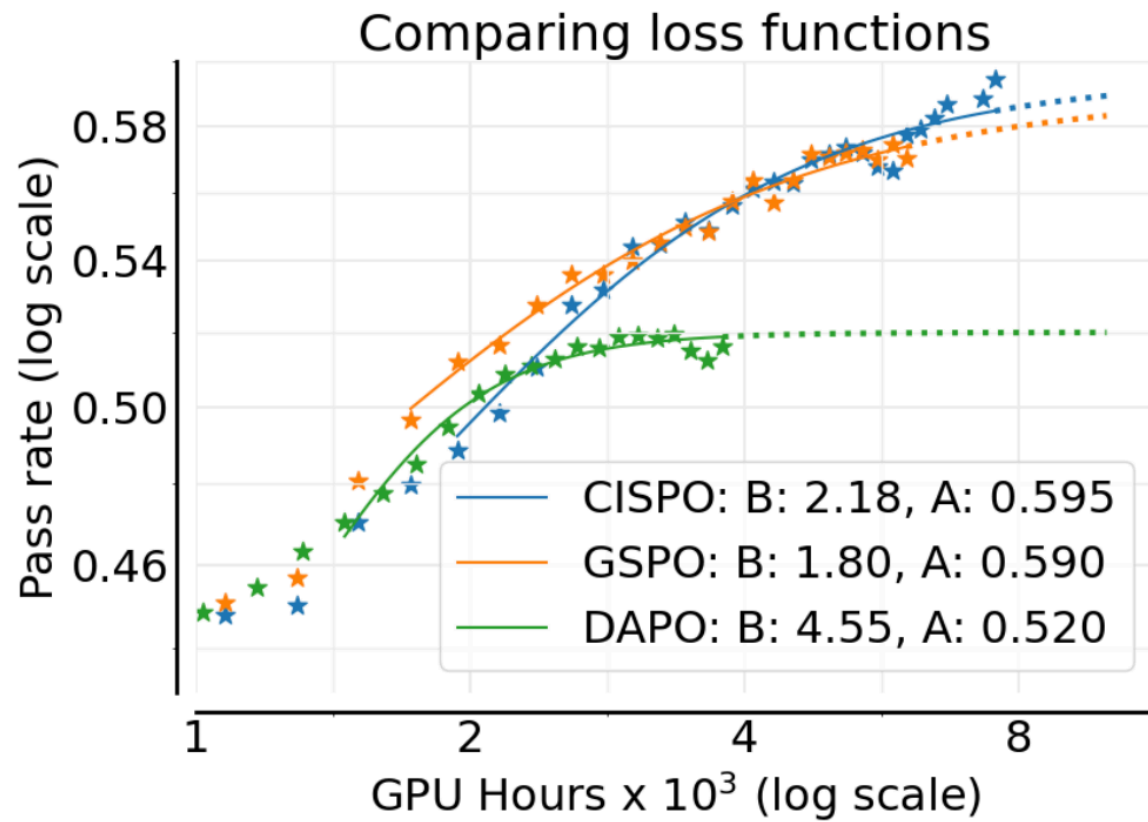
4 ScaleRL: Scaling RL Compute Effectively & Predictably

From the design axes studied above, we consolidate the best-performing settings into a single recipe, which we term **ScaleRL** (**Scale**-able **RL**). **ScaleRL** is an asynchronous RL recipe that uses **PipelineRL with 8 steps off-policyness, interruption-based length control** for truncation, **FP32 computation for logits**, and optimizes the $\mathcal{J}_{\text{ScaleRL}}(\theta)$ **loss**. This loss combines prompt-level loss aggregation, batch-level advantage normalization, **truncated importance-sampling** REINFORCE loss (CISPO) , zero-variance filtering, and no-positive resampling:

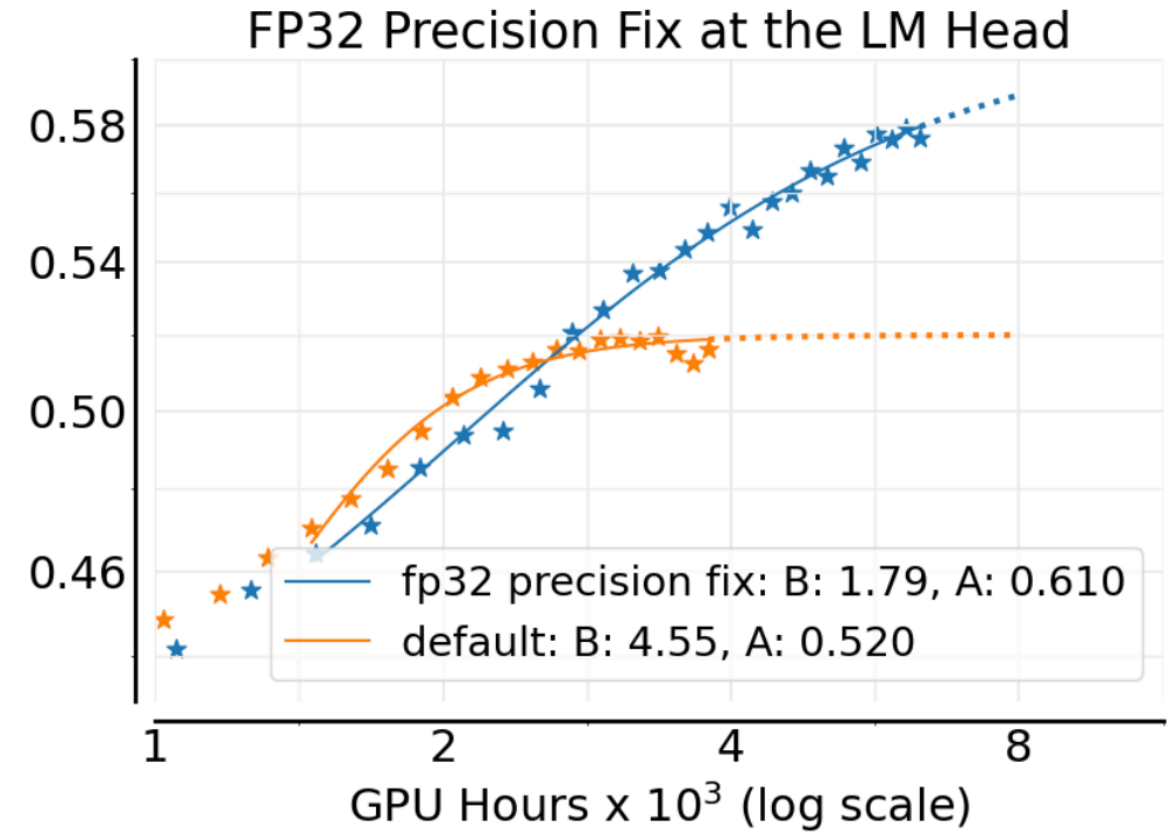
$$\mathcal{J}_{\text{ScaleRL}}(\theta) = \mathbb{E}_{\substack{x \sim D, \\ \{y_i\}_{i=1}^G \sim \pi_{\text{gen}}^{\theta_{\text{old}}}(\cdot|x)}} \left[\frac{1}{\sum_{g=1}^G |y_g|} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \text{sg}(\min(\rho_{i,t}, \epsilon)) \hat{A}_i^{\text{norm}} \log \pi_{\text{train}}^{\theta}(y_{i,t}) \right],$$
$$\rho_{i,t} = \frac{\pi_{\text{train}}^{\theta}(y_{i,t})}{\pi_{\text{gen}}^{\theta_{\text{old}}}(y_{i,t})}, \quad \hat{A}_i^{\text{norm}} = \hat{A}_i / \hat{A}_{\text{std}}, \quad 0 < \text{mean}(\{r_j\}_{j=1}^G) < 1, \quad \text{pass_rate}(x) < 0.9,$$

where **sg** is the stop-gradient function, $\hat{A}_{\text{std}}$ is the standard deviation of all advantages $\hat{A}_i$ in a batch and $\text{pass_rate}(x)$ denotes the historical pass rate of a prompt x . For forced interruptions, we use the end-of-thinking phrase: “Okay, time is up. Let me stop thinking and formulate a final answer now. `</think>`”.

ScaleRL



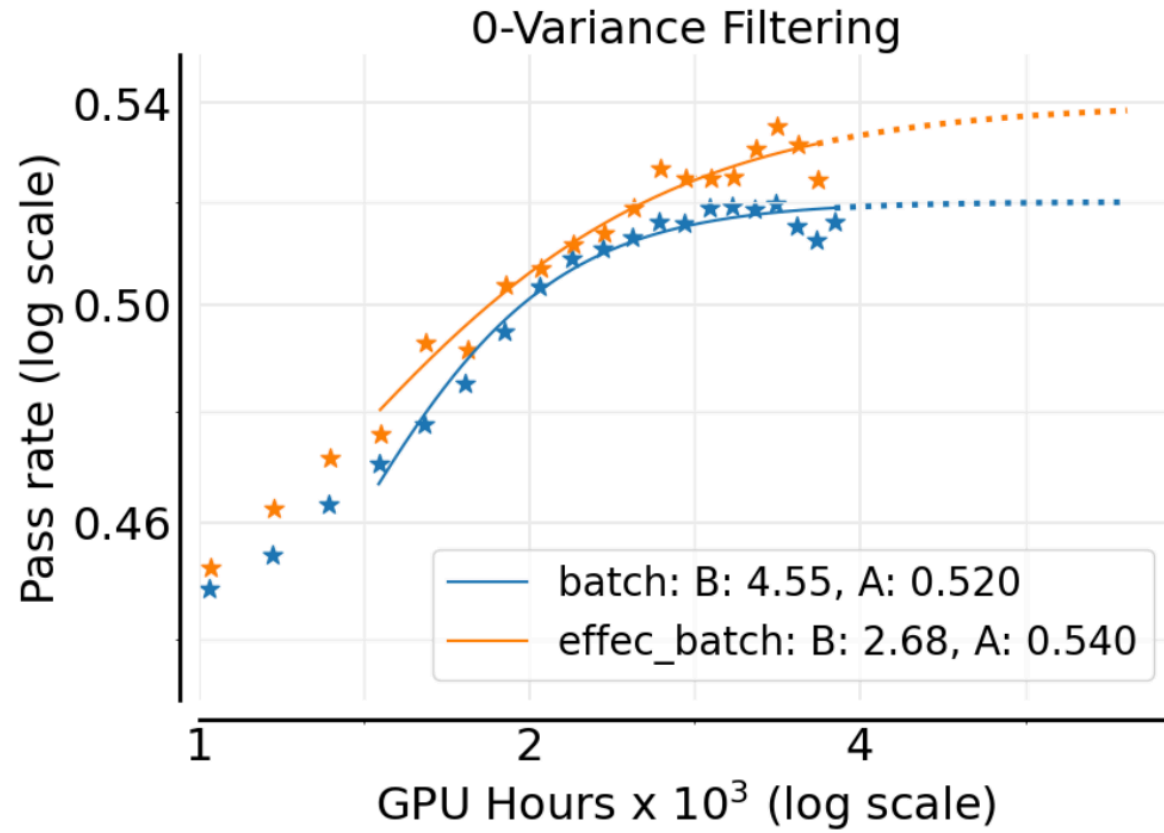
(a)



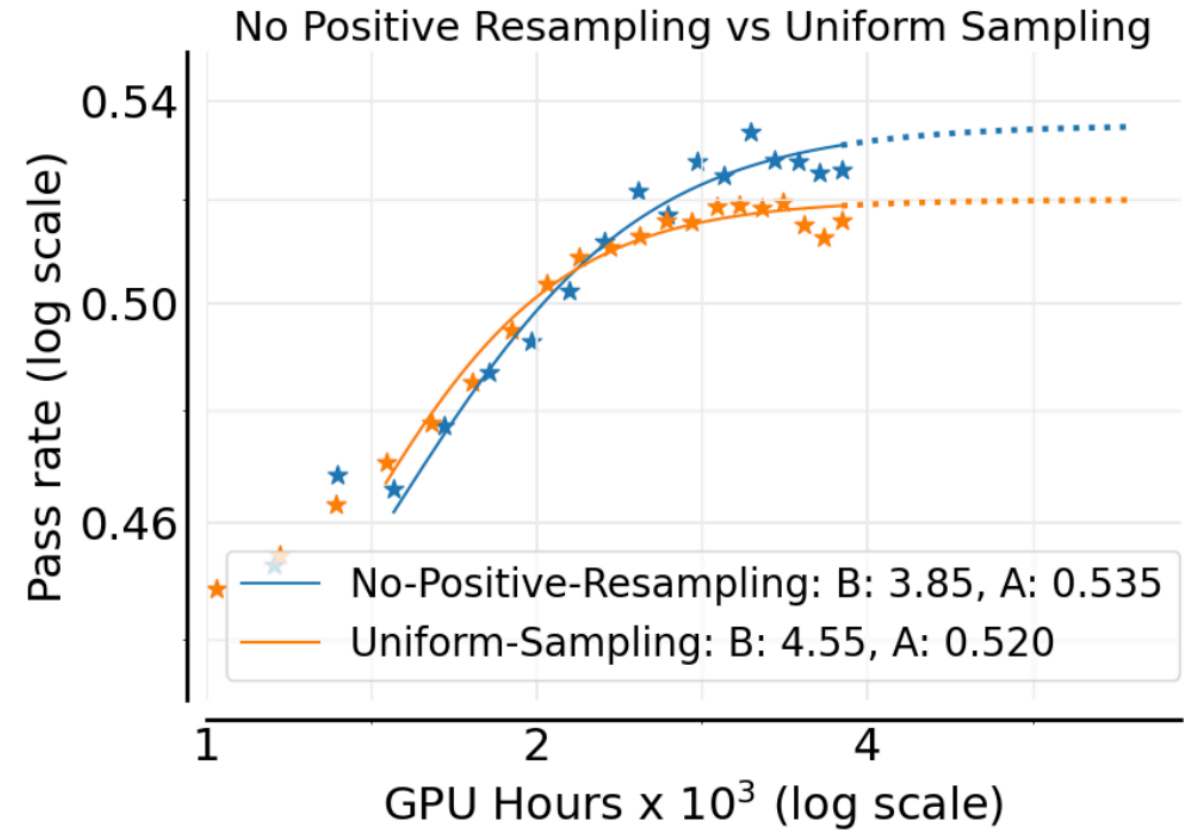
(b)

Figure 5 (a) **Comparing popular loss functions:** DAPO (Yu et al., 2025), GSPO (Zheng et al., 2025a), and CISPO (MiniMax et al., 2025). We find CISPO/GSPO achieve a higher asymptotic reward compared to DAPO. (b) **Using FP32 precision** in the final layer (LM head) gives a considerable boost in the asymptotic reward.

ScaleRL



(a)



(b)

Figure 6 (a) **“Zero” variance filtering:** We filter out “zero” variance (accuracy 0 or 1) samples in a batch since they contribute zero policy gradient and find it achieves a higher asymptote, and (b) **Adaptive prompt sampling:** Filtering out prompts with pass rate ≥ 0.9 in subsequent epochs results in a higher asymptotic performance.

ScaleRL

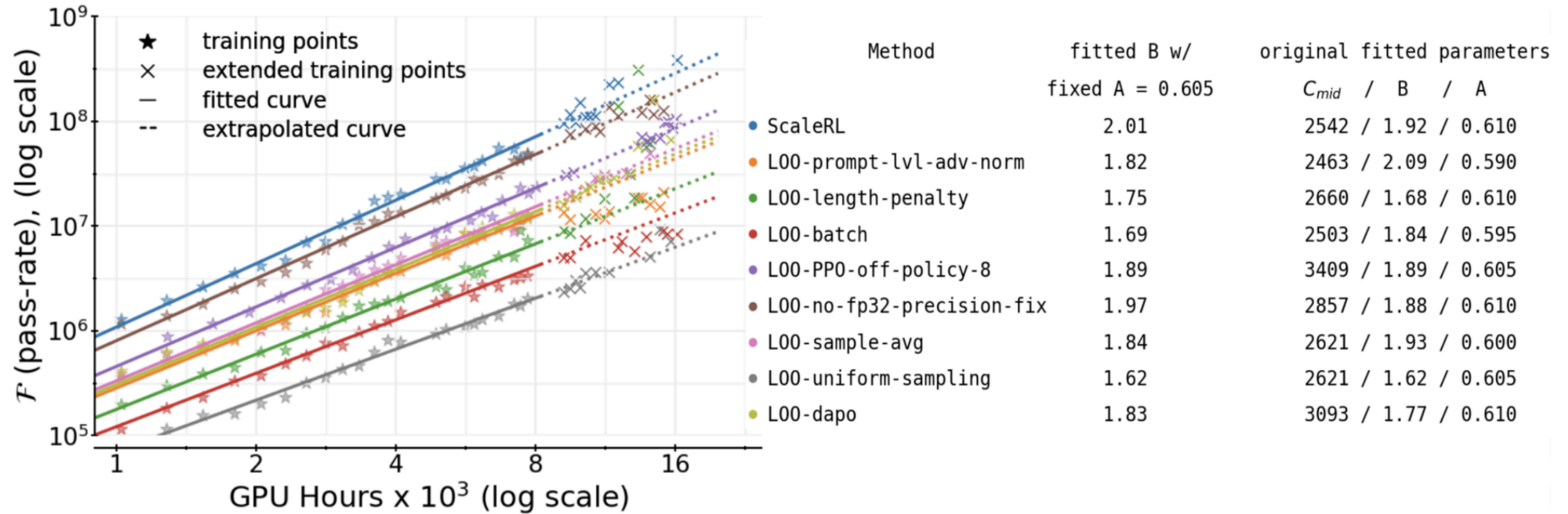


Figure 7 Leave-One-Out (LOO) Experiments: Starting from **ScaleRL**, we revert one design choice at a time to its baseline counterpart and re-train. Most LOO variants reach a similar asymptotic reward, with **ScaleRL** outperforming slightly overall. The main difference in these methods lies in efficiency. To highlight this, we re-arrange Equation (1) into $\mathcal{F}(R_c) = C^B$, where $\mathcal{F}(R_c) = C_{mid}^B / (\frac{A-R_0}{R_c-R_0} - 1)$, and plot $\log \mathcal{F}(R_c)$ vs. $\log C$. This form makes slope B directly visible, showing that **ScaleRL** achieves the highest compute efficiency.

First check if each individual component is useful; then do LOO experiments.

Only linearly many experiments.

On the Interplay of Pre-Training, Mid-Training, and RL on Reasoning Language Models

Charlie Zhang* Graham Neubig Xiang Yue†

Carnegie Mellon University, Language Technologies Institute

 [Interplay-LM-Reasoning](#)  [Interplay-LM-Reasoning](#)

{chariezhang0106, xiangyue.work}@gmail.com gneubig@cs.cmu.edu

Abstract

Recent reinforcement learning (RL) techniques have yielded impressive reasoning improvements in language models, yet it remains unclear whether post-training truly extends a model’s reasoning ability beyond what it acquires during pre-training. A central challenge is the lack of control in modern training pipelines: large-scale pre-training corpora are opaque, mid-training is often underexamined, and RL objectives interact with unknown prior knowledge in complex ways. To resolve this ambiguity, we develop a fully controlled experimental framework that isolates the causal contributions of pre-training, mid-training, and RL-based post-training. Our approach employs synthetic reasoning tasks with explicit atomic operations, parseable step-by-step reasoning traces, and systematic manipulation of training distributions. We evaluate models along two axes: *extrapolative generalization* to more complex compositions and *contextual generalization* across surface contexts. Using this framework, we reconcile competing views on RL’s effectiveness. We show that: 1) RL produces true capability gains (pass@128) only when pre-training leaves sufficient headroom and when RL data target the model’s *edge of competence*, tasks at the boundary that are difficult but not yet out of reach. 2) Contextual generalization requires minimal yet sufficient pre-training exposure, after which RL can reliably transfer. 3) Mid-training significantly enhances performance under fixed compute compared with RL only, demonstrating its central but underexplored role in training pipelines. 4) Process-level rewards reduce reward hacking and improve reasoning fidelity. Together, these results clarify the interplay between pre-training, mid-training, and RL, offering a foundation for understanding and improving reasoning LM training strategies.

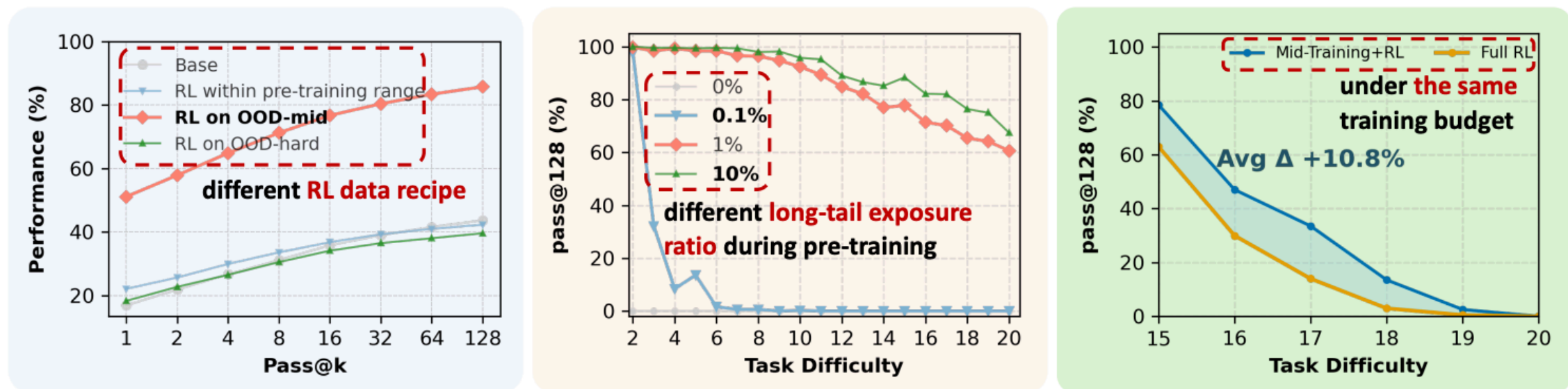


Figure 1: **Interplay of pre-, mid-, and post-training in LM reasoning.** **Left:** RL yields genuine extrapolative gains only when task difficulty slightly exceeds the pre-training range; gains vanish when tasks are already covered or too out-of-distribution (up to +42% pass@128 when well-calibrated). **Mid:** Contextual generalization requires minimal yet sufficient pre-training exposure to long-tail contexts. RL fails with near-zero exposure but generalizes robustly with sparse exposure ($\geq 1\%$), yielding up to +60% pass@128. **Right:** A mid-training stage bridging pre-training and RL substantially improves OOD reasoning under fixed compute, with mid-training + RL outperforming RL alone by +10.8% on OOD-hard tasks.

When Does Post-Training Incentivize Reasoning Beyond the Base Model?

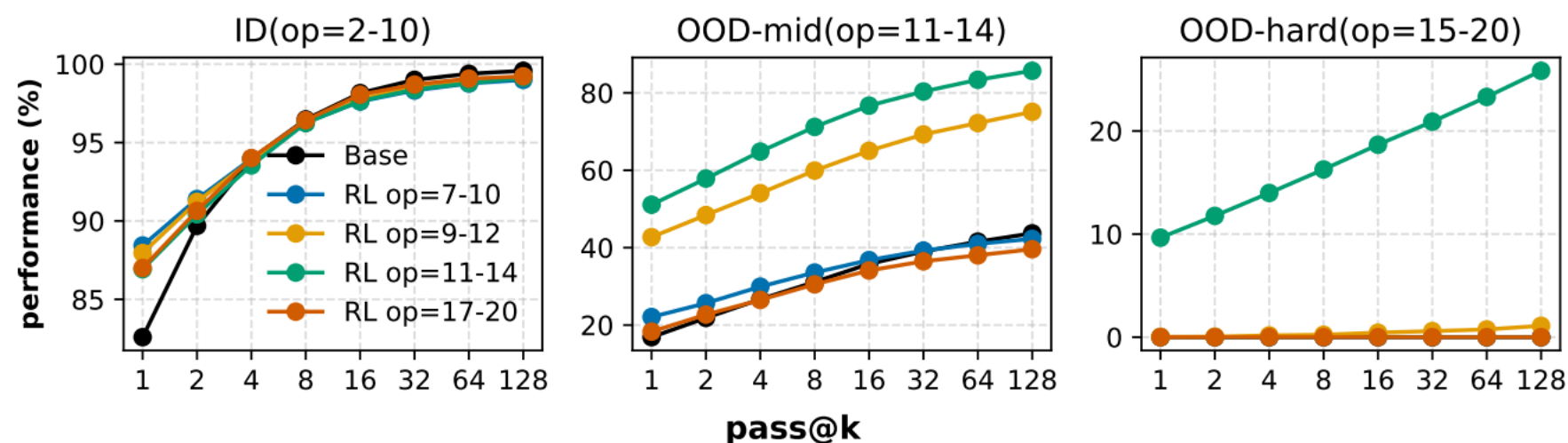


Figure 3: pass@k performance on three tasks: ID (op=2–10), OOD-edge (op=11–14), OOD-hard (op= (15–20)). RL is applied to four different data regimes (colors). RL on ID tasks never improves beyond the base model at pass@128. RL consistently improves pass@128 on harder tasks when applied beyond the base model’s capacity.

Takeaway 1

RL produces true capability gains (pass@128) beyond base models only when two conditions hold: (i) The task is not heavily covered during pre-training, leaving sufficient headroom for exploration; and (ii) the RL data is calibrated to the model’s *edge of competence*, neither too easy (in-distribution) nor too hard (out-of-distribution).

How Does Pre-training Exposure Shape Post-Training Generalization?

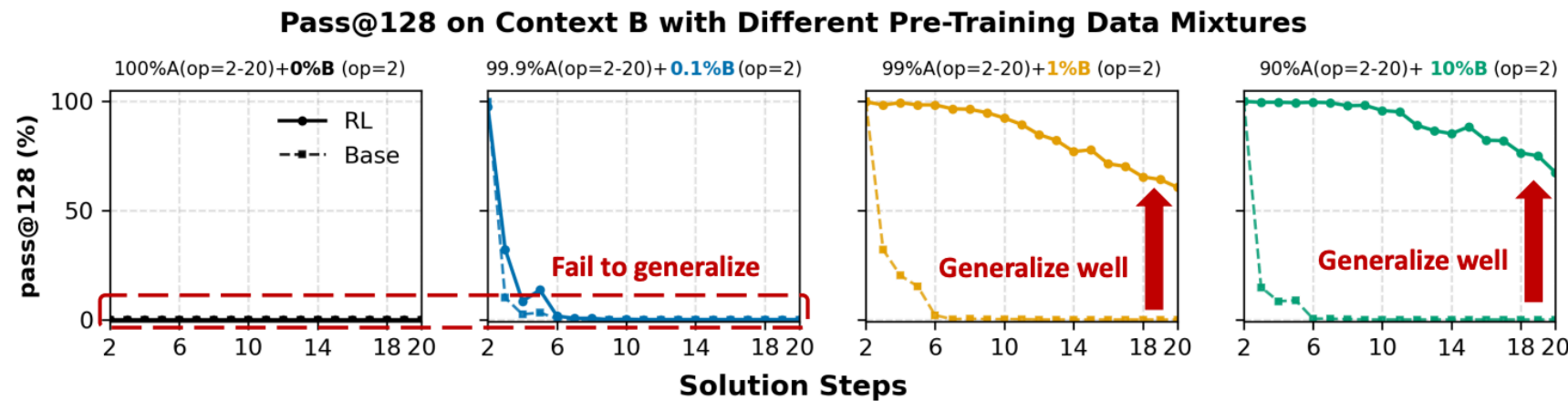


Figure 4: pass@128 performance on *context B* after post-trained with a 50% *context A* + 50% *context B* mixture. Different lines represent levels of pre-training exposure to long-tailed *context B* atomic $op=2$ examples. RL incentivizes contextual generalization when the model has minimal exposure ($\geq 1\%$) to *context B* in pre-training.

Observation 2

As shown in Figure 4, the impact of pre-training exposure to long-tailed contexts on post-training generalization is substantial: (i) When pre-training excludes *context B* or provides no (0%) or very little exposure (0.1%), RL fails to transfer to *context B*. (ii) Introducing even 1% of *context B* data during pre-training significantly enhances post-training generalization even to the hardest tasks of $op=20$. This observation underscores that while RL plays a crucial role in generalization, its effectiveness is heavily dependent on the coverage of the pre-training data, particularly the inclusion of long-tailed contexts.

Takeaway 2

RL incentivizes contextual generalization only when the base model already contains the necessary primitives. Without minimal pre-training exposure to a new context, RL cannot induce transfer. However, even sparse exposure (e.g., $\geq 1\%$) provides a sufficient seed that RL can reinforce during post-training, yielding robust cross-context generalization.

Contextual generalization: whether model transfers reasoning skills across novel surface contexts that share equivalent underlying logic

How Does Mid-Training Interact with Post-Training?

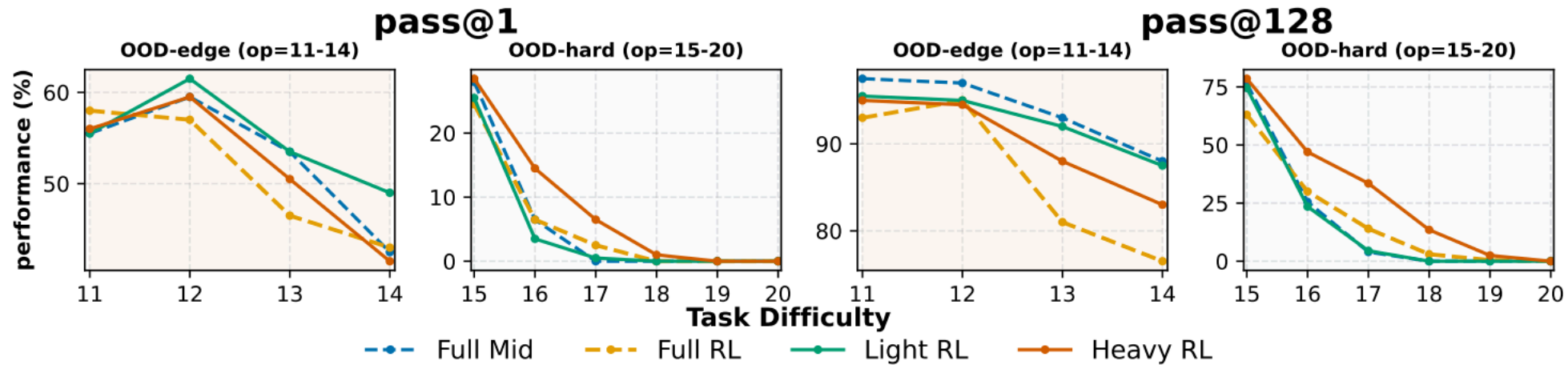


Figure 6: $\text{pass}@1$ and $\text{pass}@128$ performances on extrapolative tasks under varying mid- and post-training mixture ratios. The data used in mid- and post-training is applied within the OOD-edge ranges. Different lines indicate the compute allocation strategies. Heavy-RL always improves the unseen OOD-hard tasks, while Light-RL improves best $\text{pass}@1$ on OOD-edge tasks.

Observation 3

As shown in Figure 6, compute allocation induces qualitatively different behaviors across the generalization spectrum. (1) On **OOD-edge** tasks, configurations with *full mid-training* and *light RL* outperform those with *heavy* or *full RL*, with light RL achieving the best $\text{pass}@1$ performance. (2) For **OOD-hard** tasks, reallocating more budget toward heavy RL substantially improves performance on the hardest instances in both $\text{pass}@1$ and $\text{pass}@128$. These trends suggest that RL-driven exploration is indispensable for generalizing to harder tasks, but a substantial mid-training allocation remains critical for instilling the priors that RL can effectively exploit. We further analyze the impact of varying compute budgets in Appendix A.10.

Takeaway 3

Introducing a mid-training phase that bridges pre- and post-training distributions substantially strengthens generalization under a fixed compute budget. This highlights mid-training as an underexplored but powerful lever in training design. Compute should be allocated in a task-aware manner: (i) when prioritizing in-distribution performance, allocate more budget to mid-training with only light RL; (ii) for out-of-distribution generalization, reserve a modest portion of compute for mid-training to establish essential priors, and dedicate the remaining budget to heavier RL exploration.

How Does Mid-Training Interact with Post-Training?

Compute Budget Formulation. For fair comparison, we normalize both phases to *equivalent training tokens* based on flops. For mid-training, the consumption T_{mid} is the number of supervised tokens processed. For RL, the token-equivalent cost is approximated as:

$$T_{\text{RL}} \approx \frac{5}{3}N \cdot r \cdot L_{\text{total}}, \quad (1)$$

where N is the number of RL samples, $r = 6$ the rollout multiplicity, and $L_{\text{total}} = 2048$ the total token length⁴.

We systematically vary the RL allocation ratio $\beta \in [0, 1]$ to distribute the **total budget** T between the two phases:

$$T_{\text{mid}} = (1 - \beta) \cdot T, \quad T_{\text{RL}} = \beta \cdot T. \quad (2)$$

Mid-training: b/w pre- and post-training; typically involves high-quality or instruction-formatted data with next-token prediction or SFT objectives

Prepare a good initialization for RL;

If desired behaviors are unlikely to appear, we can waste RL compute.

References

- <https://arxiv.org/pdf/2501.12948>
- <https://arxiv.org/pdf/2503.20783>
- <https://arxiv.org/pdf/2503.14476>
- <https://arxiv.org/pdf/2505.22617>
- <https://arxiv.org/pdf/2512.16649>
- <https://arxiv.org/pdf/2509.02547>
- <https://arxiv.org/pdf/2503.09516>

References

- <https://api-docs.deepseek.com/news/news260424>
- <https://www.tuananhle.co.uk/notes/reverse-forward-kl.html>
- <https://thinkingmachines.ai/blog/on-policy-distillation/>
- <https://arxiv.org/pdf/2507.20534>
- <https://arxiv.org/pdf/2602.02276>
- <https://arxiv.org/pdf/2602.15763>
- <https://ringtech.notion.site/icepop#271c8705a032809cb68eef1c9d2ba52d>
- <https://arxiv.org/pdf/2506.13585>
- <https://arxiv.org/pdf/2510.13786>
- <https://arxiv.org/pdf/2512.07783>